

```
GetRequest-PDU ::= [0]
  IMPLICIT SEQUENCE {
    request-id
      Integer32,
    error-status
      INTEGER (0..18),
    error-index
      INTEGER (0..max-bindings),
    variable-bindings
      VarBindList
  }
```

Figure 30.10 The ASN.1 definition of a *get-request* message. Formally, the message is defined to be a *GetRequest-PDU*.

Further definitions in the standard specify the remaining undefined terms. Both *error-status* and *error-index* are single octet integers which contain the value zero in a request. If an error occurs, the values sent in a response identify the cause of the error. Finally, *VarBindList* contains a list of object identifiers for which the client seeks values. In ASN.1 terms, the definitions specify that *VarBindList* is a sequence of pairs of object name and value. ASN.1 represents the pairs as a sequence of two items. Thus, in the simplest possible request, *VarBindList* is a sequence of two items: a name and a *null*.

30.11 Example Encoded SNMP Message

The encoded form of ASN.1 uses variable-length fields to represent items. In general, each field begins with a header that specifies the type of object and its length in bytes. For example, each *SEQUENCE* begins with an octet containing 30 (hexadecimal); the next octet specifies the number of following octets that comprise the sequence.

Figure 30.11 contains an example SNMP message that illustrates how values are encoded into octets. The message is a *get-request* that specifies data item *sysDescr* (numeric object identifier *1.3.6.1.2.1.1.1.0*). Because the example shows an actual message, it includes many details. In particular, the message contains a *msgSecurityParameters* section which has not been discussed above. This particular message uses the *UsmSecurityParameters* form of security parameters. It should be possible, however, to correlate other sections of the message with the definitions above.

```

30      67      02      01      03
SEQUENCE len=103 INTEGER len=1 vers=3

30      0D      02      01      2A
SEQUENCE len=13 INTEGER len=1 msgID=42

02      02      08      00
INTEGER len=2 maxmsgsize=2048

04      01      04
string len=1 msgFlags=0x04 (bits mean noAuth, noPriv, reportable)

02      01      03
INTEGER len=1 used-based security

04      25      30      23
string len=37 SEQUENCE len=35 UseSecurityParameters

04      0C      00      00      00      63      00      00      00
string len=12 msgAuthoritativeEngineID ...

A1      C0      93      8E      23
engine is at IP address 192.147.142.35, port 161

02      01      00
INTEGER len=1 msgAuthoritativeEngineBoots=0

02      01      00
INTEGER len=1 msgAuthoritativeEngineTime=0

04      09      43      6F      6D      65      72      42      6F
string len=9 -----msgUserName value is "ComerBook"-----
6F      6B
-----

04      00
string len=0 msgAuthenticationParameters (none)

04      00
string len=0 msgPrivacyParameters (none)

30      2C
SEQUENCE len=44 ScopedPDU

04      0C      00      00      00      63      00      00
string len=12 -----contextEngineID-----
00      A1      c0      93      8E      23
-----

04      00
string len=0 contextName = "" (default)

```

CONTEXT [0] IMPLICIT SEQUENCE

```

    A0      1A
getreq. len=26

    02      02      4D      C6
INTEGER len=2   request-id = 19910

    02      01      00
INTEGER len=1 error-status = noError(0)

    02      01      00
INTEGER len=1 error-index=0

    30      0E
SEQUENCE len=14 VarBindList

    30      0C
SEQUENCE len=12 VarBind

    06      08
OBJECT IDENTIFIER name

    2B      06      01      02      01      01      01      00
1.3 . 6 . 1 . 2 . 1 . 1 . 1 . 0 (sysDescr.0)

    05      00
null len=0 (no value specified)

```

Figure 30.11 The encoded form of an SNMPv3 *get-request* for data item *sysDescr* with octets shown in hexadecimal and a comment explaining their meaning below. Related octets have been grouped onto lines; they are contiguous in the message.

As Figure 30.11 shows, the message starts with a code for *SEQUENCE* which has a length of 103 octets†. The first item in the sequence is a 1-octet integer that specifies the protocol *version*; the value 3 indicates that this is an SNMPv3 message. Successive fields define a message ID and the maximum message size the sender can accept in a reply. Security information, including the name of the user (*ComerBook*) follows the message header.

The *GetRequest-PDU* occupies the tail of the message. The sequence labeled *ScopedPDU* specifies a context in which to interpret the remainder of the message. The octet *A0* specifies the operation as a *get-Request*. Because the high-order bit is turned on, the interpretation of the octet is *context specific*. That is, the hexadecimal value *A0* only specifies a *GetRequest-PDU* when used in context; it is not a universally reserved value. Following the request octet, the length octet specifies the request is 26 octets long. The request ID is 2 octets, but each of the error-status and error-index are one oc-

†Sequence items occur frequently in an SNMP message because SNMP uses *SEQUENCE* instead of conventional programming language constructs like *array* or *struct*.

tet. Finally, the sequence of pairs contains one binding, a single object identifier bound to a *null* value. The identifier is encoded as expected except that the first two numeric labels are combined into a single octet.

30.12 New Features In SNMPv3

We said that version 3 of SNMP represents an evolution that follows and extends the basic framework of earlier versions. The primary changes arise in the areas of security and administration. The goals are twofold. First, SNMPv3 is designed to have both general and flexible security policies, making it possible for the interactions between a manager and managed devices to adhere to the security policies an organization specifies. Second, the system is designed to make administration of security easy.

To achieve generality and flexibility, SNMPv3 includes facilities for several aspects of security, and allows each to be configured independently. For example, v3 supports *message authentication* to ensure that instructions originate from a valid manager, *privacy* to ensure that no one can read messages as they pass between a manager's station and a managed device, and *authorization* and *view-based access control* to ensure that only authorized managers access particular items. To make the security system easy to configure or change, v3 allows *remote configuration*, meaning that an authorized manager can change the configuration of security items listed above without being physically present at the device.

30.13 Summary

Network management protocols allow a manager to monitor and control routers and hosts. A network management client program executing on the manager's workstation contacts one or more servers, called agents, running on the devices to be controlled. Because an internet consists of heterogeneous machines and networks, TCP/IP management software executes as application programs and uses internet transport protocols (e.g., UDP) for communication between clients and servers.

The standard TCP/IP network management protocol is SNMP, the Simple Network Management Protocol. SNMP defines a low-level management protocol that provides two conceptual operations: fetch a value from a variable or store a value into a variable. In SNMP, other operations occur as side-effects of changing values in variables. SNMP defines the format of messages that travel between a manager's computer and a managed entity.

A set of companion standards to SNMP define the set of variables that a managed entity maintains. The set of variables comprise a Management Information Base (*MIB*). MIB variables are described using ASN.1, a formal language that provides a concise encoded form as well as a precise human-readable notation for names and objects. ASN.1 uses a hierarchical namespace to guarantee that all MIB names are globally unique while still allowing subgroups to assign parts of the namespace.

FOR FURTHER STUDY

Case et. al. [RFC 2570] presents an overview of SNMPv3, gives background and motivation, and discusses changes among the various versions. It also contains a summary of RFCs related to v3, and explains which v2 standards still apply. Many other RFCs discuss individual aspects of the protocol. For example, Wijnen et. al. [RFC 2575] presents the view-based access control model, and Case et. al. [RFC 2572] discusses message handling.

ISO [May 87a] and [May 87b] contain the standard for ASN.1 and specify the encoding. McCloghrie et. al. [RFCs 2578, 2579, 2580] define the language used for MIB modules and provide definitions of data types. Case et. al. [RFC 1907] defines version 2 of the MIB.

An older proposal for a network management protocol called HEMS can be found in Trewitt and Partridge [RFCs 1021, 1022, 1023, and 1024]. Davin, Case, Fedor, and Schoffstall [RFC 1028] specifies a predecessor to SNMP known as the Simple Gateway Monitoring Protocol (*SGMP*).

EXERCISES

- 30.1** Capture an SNMP packet with a network analyzer and decode the fields.
- 30.2** Read the standard to find out how ASN.1 encodes the first two numeric values from an object identifier in a single octet. Why does it do so?
- 30.3** Read the two standards and compare SNMPv2 to SNMPv3. Under what circumstances are the v2 security features valid? Invalid?
- 30.4** Suppose the MIB designers need to define a variable that corresponds to a two-dimensional array. How can ASN.1 notation accommodate references to such a variable?
- 30.5** What are the advantages and disadvantages of defining globally unique ASN.1 names for MIB variables?
- 30.6** Consult the standards and match each item in Figure 30.11 with a corresponding definition.
- 30.7** If you have SNMP client code available, try using it to read MIB variables in a local router. What is the advantage of allowing arbitrary managers to read variables in all routers? The disadvantage?
- 30.8** Read the MIB specification to find the definition of variable *ipRoutingTable* that corresponds to an IP routing table. Design a program that will use SNMP to contact multiple routers and see if any entries in their routing tables cause a routing loop. Exactly what ASN.1 names should such a program generate?
- 30.9** Consider the implementation of an SNMP agent. Does it make sense to arrange MIB variables in memory exactly the way SNMP describes them? Why or why not?

- 30.10** Argue that SNMP is a misnomer because SNMP is not “simple.”
- 30.11** Read about the IPsec security standard described in Chapter 32. If an organization uses IPsec, is the security in SNMPv3 also necessary? Why or why not?
- 30.12** Does it make sense to use SNMP to manage all devices? Why or why not? (Hint: consider a simple hardware device such as a dialup modem.)

Summary Of Protocol Dependencies

31.1 Introduction

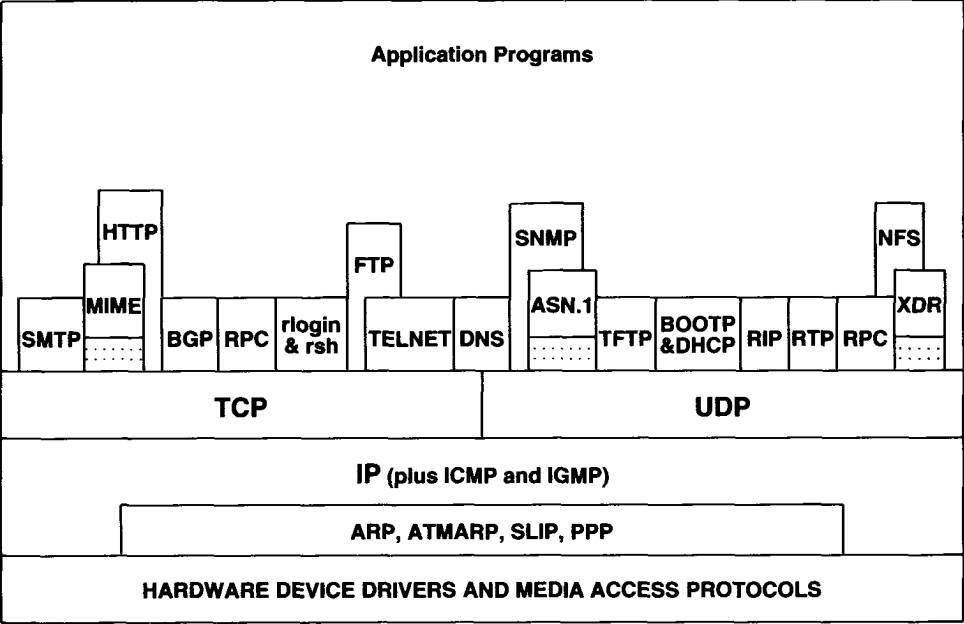
TCP/IP has spawned more applications than we can discuss in a single textbook. In general, each defines its own application protocol and relies on TCP or UDP for end-to-end transport. In fact, any programmer who builds a distributed application using TCP/IP defines an application-level protocol.

Although it is not important to understand the details of all protocols, it is important to know which protocols exist and how they can be used. This chapter provides a brief summary of the relationships among fundamental protocols, and shows which are available for use by applications.

31.2 Protocol Dependencies

The chart in Figure 31.1 shows dependencies among the major protocols we have discussed. Each enclosed polygon corresponds to one protocol, and resides directly above protocols that it uses. For example, the mail protocol, SMTP, depends on TCP, which depends on IP.

Users



Hardware

Figure 31.1 Dependencies among major, higher level TCP/IP protocols. A protocol uses the protocols that lie directly below it. Application programs can use all protocols above IP.

Several parts of the diagram need further explanation. The bottom layer represents all protocols that the hardware provides. This level includes all hardware control protocols (e.g., media access and logical link allocation). As we have throughout the text, we will assume that any packet transfer system can be included in this layer as long as IP can use it to transfer datagrams. Thus, if a system is configured to send datagrams through a tunnel, the entry to the tunnel is treated like a hardware interface, despite its software implementation.

The second layer from the bottom lists link layer and address resolution protocols like SLIP, PPP, ARP, and ATMARF. Of course, not all networking technologies require such protocols. ARP is used on connectionless broadcast networks such as Ethernet; ATMARF is used on non-broadcast multiple access networks such as ATM; and RARP is seldom used except for diskless machines. Other link layer or address binding protocols can occur at the same level, but none is currently popular.

The third layer from the bottom contains IP. It includes the required error and control message protocol, ICMP, and the optional multicast group management protocol IGMP. Note that IP is the only protocol that spans an entire layer. All lower-level protocols deliver incoming information to IP, and all higher-level protocols must use IP to send outgoing datagrams. IP is shown with a direct dependency on the hardware layer because it needs to use the hardware link or access protocols to transmit datagrams after it uses ARP to bind addresses.

TCP and UDP comprise the transport layer. Of course, new transport protocols have been suggested, but none has been widely adopted yet.

The application layer illustrates the complex dependencies among the various application protocols. Recall, for example, that FTP uses the network virtual terminal definitions from TELNET to define communication on its control connection and TCP to form data connections. Also recall that HTTP uses syntax and content types from MIME. Thus, the diagram shows that FTP depends on both TELNET and TCP and that HTTP depends on both MIME and TCP. The domain name system (DNS) uses both UDP and TCP for communication, so the diagram shows both dependencies. Sun's NFS depends on the external data representation (XDR) and remote procedure call (RPC) protocols. RPC appears twice because, like the domain name system, it can use either UDP or TCP.

SNMP depends on *Abstract Syntax Notation* (ASN.1). Although SNMP can use either UDP or TCP, only dependence on UDP is shown because few implementations run over TCP. Because XDR, ASN.1, and MIME simply describe syntactic conventions and data representations, they do not use either TCP or UDP. Thus, although it shows that both SNMP and NFS depend on UDP, the diagram contains a dotted area below ASN.1 and XDR to show that neither of them depends on UDP. A few details have been omitted in our diagram. For example, it could be argued that IP depends on BOOTP/DHCP or that many protocols depend on DNS because software that implements such protocols requires name binding.

31.3 The Hourglass Model

Engineers describe Internet protocols as following an *hourglass model*. Because it lies at the heart of all communication, IP forms the center of the hourglass. Of all the protocols we discussed, IP is the only protocol common to all applications — ultimately all internet communication involves IP datagrams. Thus, universal interoperability is achieved by making IP run over all possible network technologies. Figure 31.2 illustrates the concept by showing the dependency among IP, applications, and underlying networks.

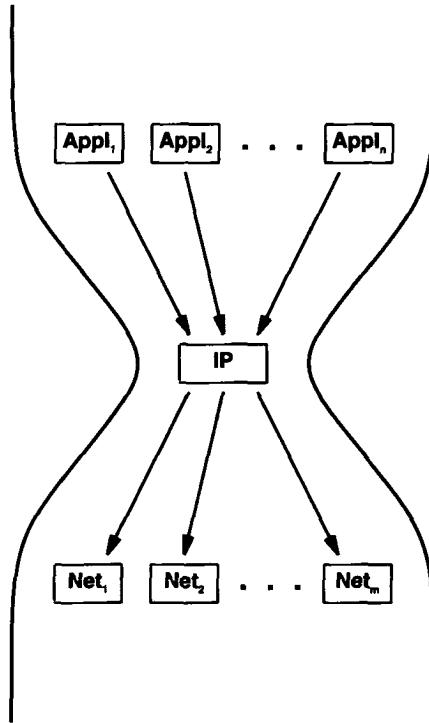


Figure 31.2 An illustration of the hourglass model. IP is at the center of the hourglass because all applications depend on IP and IP runs over all networks.

31.4 Application Program Access

Most systems restrict application programs from accessing lower-level protocols. That is, most systems only allow an application program to access TCP or UDP or to implement higher level protocols that use them (e.g., FTP). In fact, a system may choose to restrict access to transport protocols by allowing only privileged applications to open lower numbered TCP or UDP protocol ports.

Although direct access from an application to IP is unusual, a few systems do provide special purpose mechanisms that make it possible. For example, a mechanism known as a *packet filter* allows privileged programs to control frame demultiplexing. Using the packet filter primitives, an application program establishes the criteria used to capture packets (e.g., the application program can specify that it wishes to capture all packets with a given value in the *type* field of a frame). Once the operating system accepts the filter command, it places all packets that match the specified type on a queue. The application program uses the packet filter mechanism to extract packets from the

queue. For such systems, the diagram in Figure 31.1 should be extended to show application access to lower layers.

31.5 Summary

Much of the rich functionality associated with the TCP/IP protocol suite results from a variety of high-level services supplied by application programs. The high-level protocols these programs use build on the basic transport services: unreliable datagram delivery and reliable stream transport. The applications usually follow the client-server model in which servers operate at known protocols ports so clients know how to contact them.

The highest level of protocols provides user services like Web browsing, remote login, and file and mail transfer. The chief advantages of having an internet on which to build such services are that it provides universal connectivity and simplifies the application protocols. In particular, when used by two machines that attach to an internet, end-to-end transport protocols can guarantee that a client program on the source machine communicates directly with a server on the destination machine. Because services like electronic mail use the end-to-end transport connection, they do not need to rely on intermediate machines to forward (whole) messages.

We have seen a variety of application level protocols and the complex dependencies among them. Although many application protocols have been defined, a few major applications such as Web browsing account for most packets on the Internet.

FOR FURTHER STUDY

One of the issues underlying protocol layering revolves around the optimal location of protocol functionality. Edge [1979] compares end-to-end protocols with the hop-by-hop approach. Saltzer, Reed, and Clark [1984] argues for having the highest level protocols perform end-to-end acknowledgement and error detection. A series of papers by Mills [RFCs 956, 957, and 958] proposes application protocols for clock synchronization, and report on experiments.

EXERCISES

- 31.1** It is possible to translate some application protocols into others. For example, it might be possible to build a program that accepts an FTP request, translates it to a TFTP request, passes the result to a TFTP server to obtain a file, and translates the reply back to FTP for transmission to the original source. What are the advantages and disadvantages of such protocol translation?

- 31.2** Consider the translation described in the previous question. Which pairs of protocols in Figure 31.1 are amenable to such translations?
- 31.3** Some application programs invoked by users may need access to IP without using TCP or UDP. Find examples of such programs. (Hint: think of ICMP.)
- 31.4** Where do multicast protocols fit into the diagram in Figure 31.1?
- 31.5** DNS allows access by both TCP and UDP. Find out whether your local operating system allows a single process to accept both TCP connections and UDP requests.
- 31.6** Choose a complex application like the *X window system*, and find out which protocols it uses.
- 31.7** Where does OSPF fit into the diagram in Figure 31.1?
- 31.8** The diagram in Figure 31.1 shows that FTP depends on TELNET. Does your local FTP client invoke the TELNET program, or does the FTP client contain a separate implementation of the TELNET protocol?
- 31.9** Redraw Figure 31.1 for a Web browser. Which protocols does it use?

Internet Security And Firewall Design (IPsec)

32.1 Introduction

Like the locks used to help keep tangible property secure, computers and data networks need provisions that help keep information secure. Security in an internet environment is both important and difficult. It is important because information has significant value — information can be bought and sold directly or used indirectly to create new products and services that yield high profits. Security in an internet is difficult because security involves understanding when and how participating users, computers, services, and networks can trust one another as well as understanding the technical details of network hardware and protocols. Security is required on every computer and every protocol; a single weakness can compromise the security of an entire network. More important, because TCP/IP supports a wide diversity of users, services, and networks and because an internet can span many political and organizational boundaries, participating individuals and organizations may not agree on a level of trust or policies for handling data.

This chapter considers two fundamental techniques that form the basis for internet security: perimeter security and encryption. Perimeter security allows an organization to determine the services and networks it will make available to outsiders and the extent to which outsiders can use resources. Encryption handles most other aspects of security. We begin by reviewing a few basic concepts and terminology.

32.2 Protecting Resources

The terms *network security* and *information security* refer in a broad sense to confidence that information and services available on a network cannot be accessed by unauthorized users. Security implies safety, including assurance of data integrity, freedom from unauthorized access of computational resources, freedom from snooping or wiretapping, and freedom from disruption of service. Of course, just as no physical property is absolutely secure against crime, no network is completely secure. Organizations make an effort to secure networks for the same reason they make an effort to secure buildings and offices: basic security measures can discourage crime by making it significantly more difficult.

Providing security for information requires protecting both physical and abstract resources. Physical resources include passive storage devices such as disks and CD-ROMs as well as active devices such as users' computers. In a network environment, physical security extends to the cables, bridges, and routers that comprise the network infrastructure. Indeed, although physical security is seldom mentioned, it often plays an important role in an overall security plan. Obviously, physical security can prevent wiretapping. Good physical security can also eliminate sabotage (e.g., disabling a router to cause packets to be routed through an alternative, less secure path).

Protecting an abstract resource such as information is usually more difficult than providing physical security because information is elusive. Information security encompasses many aspects of protection:

- *Data integrity.* A secure system must protect information from unauthorized change.
- *Data availability* The system must guarantee that outsiders cannot prevent legitimate access to data (e.g., any outsider should not be able to block customers from accessing a Web site).
- *Privacy or confidentiality.* The system must prevent outsiders from making copies of data as it passes across a network or understanding the contents if copies are made.
- *Authorization.* Although physical security often classifies people and resources into broad categories, (e.g., all nonemployees are forbidden from using a particular hallway), security for information usually needs to be more restrictive (e.g., some parts of an employee's record are available only to the personnel office, others are available only to the employee's boss, and others are available to the payroll office).
- *Authentication.* The system must allow two communicating entities to validate each other's identity.
- *Replay avoidance.* To prevent outsiders from capturing copies of packets and using them later, the system must prevent a retransmitted copy of a packet from being accepted.

32.3 Information Policy

Before an organization can enforce network security, the organization must assess risks and develop a clear policy regarding information access and protection. The policy specifies who will be granted access to each piece of information, the rules an individual must follow in disseminating the information to others, and a statement of how the organization will react to violations.

An information policy begins with people because:

Humans are usually the most susceptible point in any security scheme. A worker who is malicious, careless, or unaware of an organization's information policy can compromise the best security.

32.4 Internet Security

Internet security is difficult because datagrams traveling from source to destination often pass across many intermediate networks and through routers that are not owned or controlled by either the sender or the recipient. Thus, because datagrams can be intercepted or compromised, the contents cannot be trusted. As an example, consider a server that attempts to use *source authentication* to verify that requests originated from valid customers. Source authentication requires the server to examine the source IP address on each incoming datagram, and only accept requests from computers on an authorized list. Source authentication is *weak* because it can be broken easily. In particular, an intermediate router can watch traffic traveling to and from the server, and record the IP address of a valid customer. Later the intermediate router can manufacture a request that has the same source address (and intercept the reply). The point is:

An authorization scheme that uses a remote machine's IP address to authenticate its identity does not suffice in an unsecure internet. An imposter who gains control of an intermediate router can obtain access by impersonating an authorized client.

Stronger authentication requires *encryption*. To encrypt a message, the sender applies a mathematical function that rearranges the bits according to a *key* which is known only to the sender. The receiver uses another mathematical function to decrypt the message. Careful choices of an encryption algorithm, a key, and the contents of messages can make it virtually impossible for intermediate machines to decode messages or manufacture messages that are valid.

32.5 IP Security (IPsec)

The IETF has devised a set of protocols that provide secure Internet communication. Collectively known as *IPsec* (short for *IP security*), the protocols offer authentication and privacy services at the IP layer, and can be used with both IPv4 and IPv6†. More important, instead of completely specifying the functionality or the encryption algorithm to be used, the IETF chose to make the system both flexible and extensible. For example, an application that employs IPsec can choose whether to use an authentication facility that validates the sender or to use an encryption facility that also ensures the payload will remain confidential; the choices can be asymmetric (e.g., authentication in one direction but not the other). Furthermore, IPsec does not restrict the user to a specific encryption or authentication algorithm. Instead, IPsec provides a general framework that allows each pair of communicating endpoints to choose algorithms and parameters (e.g., key size). To guarantee interoperability, IPsec does include a set of encryption algorithms that all implementations must recognize. The point is:

IPsec is not a single security protocol. Instead, IPsec provides a set of security algorithms plus a general framework that allows a pair of communicating entities to use whichever algorithms provide security appropriate for the communication.

32.6 IPsec Authentication Header

Instead of changing the basic datagram header or creating an IP option, IPsec uses a separate *Authentication Header (AH)* to carry authentication information. Figure 32.1 illustrates the most straightforward use of an authentication header with IPv4.

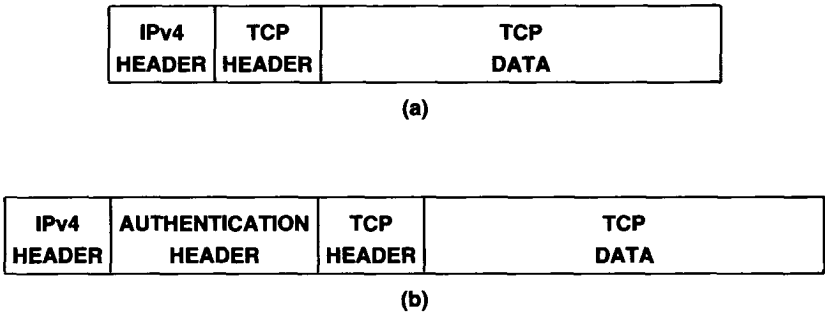


Figure 32.1 Illustration of (a) an IPv4 datagram, and (b) the same datagram after an IPsec authentication header has been added. The new header is inserted immediately after the IP header.

†The examples in this chapter focus on IPv4; Chapter 33 describes IPv6 in detail and illustrates how IPsec headers appear in IPv6 datagrams.

As the figure shows, IPsec inserts the authentication header immediately after the original IP header, but before the transport header. Furthermore, the *PROTOCOL* field in the IP header is changed to value 51 to indicate the presence of an authentication header.

If IPsec modifies the *PROTOCOL* field in the IP header, how does a receiver determine the type of information carried in the datagram? The authentication header has a *NEXT HEADER* field that specifies the type — IPsec records the original *PROTOCOL* value in the *NEXT HEADER* field. When a datagram arrives, the receiver uses security information from the authentication header to verify the sender, and uses the *NEXT HEADER* value to further demultiplex the datagram. Figure 32.2 illustrates the header format.

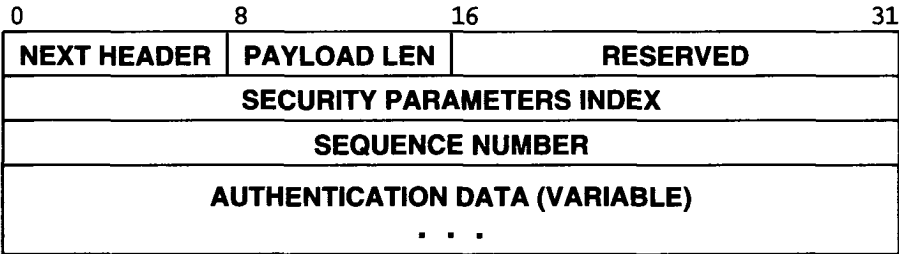


Figure 32.2 The IPsec authentication header format. The field labeled *NEXT HEADER* records the original value of the IP *PROTOCOL* field.

Interestingly, the *PAYLOAD LEN* field does not specify the size of the data area in the datagram. Instead, it specifies the length of the authentication header. Remaining fields are used to ensure security. Field *SEQUENCE NUMBER* contains a unique sequence number for each packet sent; the number starts at zero when a particular security algorithm is selected and increases monotonically. The *SECURITY PARAMETERS INDEX* field specifies the security scheme used, and the *AUTHENTICATION DATA* field contains data for the selected security scheme.

32.7 Security Association

To understand the reason for using a security parameters index, observe that a security scheme defines details that provide many possible variations. For example, the security scheme includes an authentication algorithm, a key (or keys) that the algorithm uses, a lifetime over which the key will remain valid, a lifetime over which the destination agrees to use the algorithm, and a list of source addresses that are authorized to use the scheme. Further observe that the information cannot fit into the header.

To save space in the header, IPsec arranges for each receiver to collect all the details about a security scheme into an abstraction known as a *security association (SA)*.

Each SA is given a number, known as a *security parameters index*, through which it is identified. Before a sender can use IPsec to communicate with a receiver, the sender must know the index value for a particular SA. The sender then places the value in the field *SECURITY PARAMETERS INDEX* of each outgoing datagram.

Index values are not globally specified. Instead, each destination creates as many SAs as it needs, and assigns an index value to each. The destination can specify a life-time for each SA, and can reuse index values once an SA becomes invalid. Consequently, the index cannot be interpreted without consulting the destination (e.g., the index 1 can have entirely different meanings to two destinations). To summarize:

A destination uses the security parameters index to identify the security association for a packet. The values are not global; a combination of destination address and security parameters index is needed to identify an SA.

32.8 IPsec Encapsulating Security Payload

To handle privacy as well as authentication, IPsec uses an *Encapsulating Security Payload (ESP)*, which is more complex than an authentication header. A value 50 in the *PROTOCOL* field of the datagram informs a receiver that the datagram carries ESP. Figure 32.3 illustrates the conceptual organization.

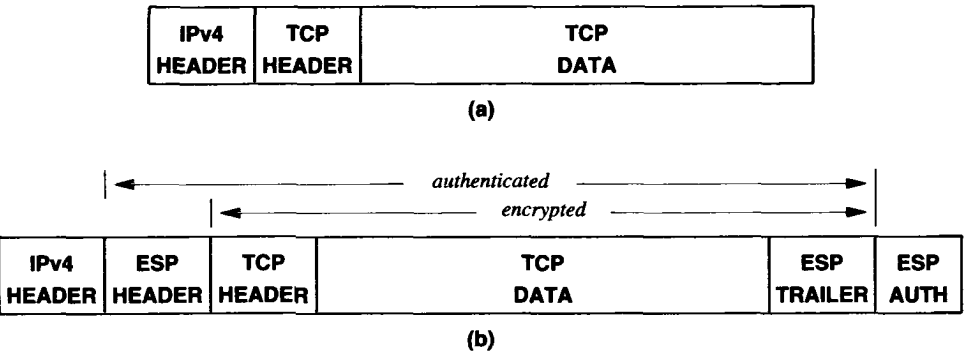
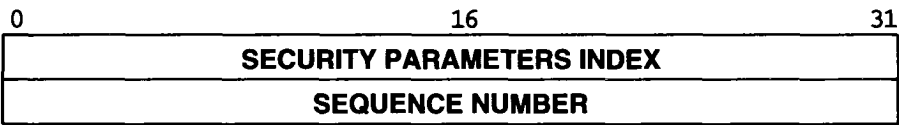


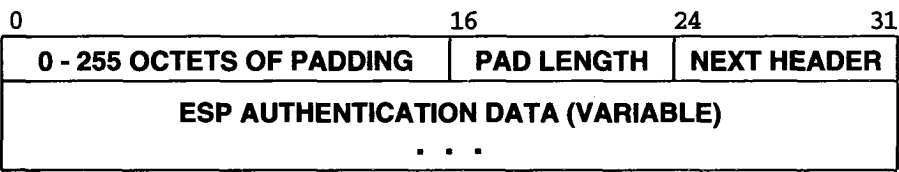
Figure 32.3 (a) A datagram, and (b) the same datagram using IPsec Encapsulating Security Payload. In practice, encryption means that fields are not easily identifiable.

As the figure shows, ESP adds three additional areas to the datagram. The *ESP HEADER* immediately follows the IP header and precedes the encrypted payload. The *ESP TRAILER* is encrypted along with the payload; a variable-size *ESP AUTH* field follows the encrypted section.

ESP uses many of the same items found in the authentication header, but rearranges their order. For example, the *ESP HEADER* consists of 8 octets that identify the security parameters index and a sequence number.



The *ESP TRAILER* consists of optional padding, a padding length field, *PAD LENGTH*, and a *NEXT HEADER* field that is followed by a variable amount of authentication data.



Padding is optional; it may be present for three reasons. First, some decryption algorithms require zeroes following an encrypted message. Second, note that the *NEXT HEADER* field is shown right-justified within a 4-octet field. The alignment is important because IPsec requires the authentication data that follows the trailer to be aligned at the start of a 4-octet boundary. Thus, padding may be needed to ensure alignment. Third, some sites may choose to add random amounts of padding to each datagram so eavesdroppers at intermediate points along the path cannot use the size of a datagram to guess its purpose.

32.9 Authentication And Mutable Header Fields

The IPsec authentication mechanism is designed to ensure that an arriving datagram is identical to the datagram sent by the source. However, such a guarantee is impossible to make. To understand why, recall that IP is a machine-to-machine layer, meaning that the layering principle only applies across one hop. In particular, each intermediate router decrements the time-to-live field and recomputes the checksum.

IPsec uses the term *mutable fields* to refer to IP header fields that are changed in transit. To prevent such changes causing authentication errors, IPsec specifically omits such fields from the authentication computation. Thus, when a datagram arrives, IPsec only authenticates immutable fields (e.g., the source address and protocol type).

32.10 IPsec Tunneling

Recall from Chapter 20 that VPN technology uses encryption along with IP-in-IP tunneling to keep inter-site transfers private. IPsec is specifically designed to accommodate an encrypted tunnel. In particular, the standard defines tunneled versions of both the authentication header and the encapsulating security payload. Figure 32.4 illustrates the layout of datagrams in tunneling mode.

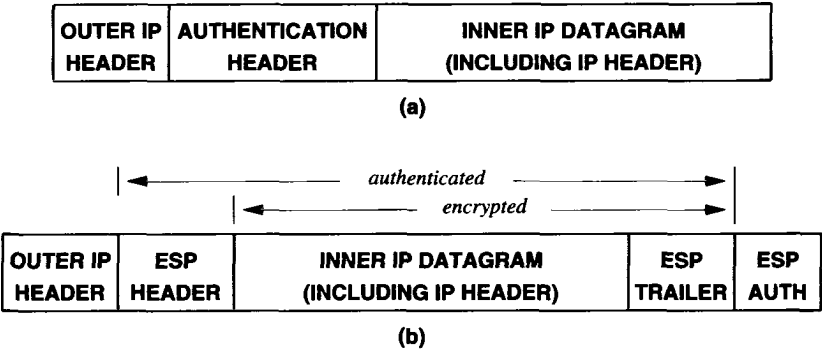


Figure 32.4 Illustration of IPsec tunneling mode for (a) authentication and (b) encapsulating security payload. The entire inner datagram is protected.

32.11 Required Security Algorithms

IPsec defines a minimal set of algorithms that are mandatory (i.e., that all implementations must supply). In each case, the standard defines specific uses. Figure 32.5 lists the required algorithms.

Authentication	
HMAC with MD5	RFC 2403
HMAC with SHA-1	RFC 2404
Encapsulating Security Payload	
DES in CBC mode	RFC 2405
HMAC with MD5	RFC 2403
HMAC with SHA-1	RFC 2404
Null Authentication	
Null Encryption	

Figure 32.5 The security algorithms that are mandatory for IPsec.

32.12 Secure Sockets

By the mid 1990s when it became evident that security was important for Internet commerce, several groups proposed security mechanisms for use with the Web. Although not formally adopted by the IETF, one of the proposals has become a de facto standard.

Known as the *Secure Sockets Layer (SSL)*, the technology was originally developed by Netscape, Inc. As the name implies, SSL resides at the same layer as the socket API. When a client uses SSL to contact a server, the SSL protocol allows each side to authenticate itself to the other. The two sides then negotiate to select an encryption algorithm that they both support. Finally, SSL allows the two sides to establish an encrypted connection (i.e., a connection that uses the chosen encryption algorithm to guarantee privacy).

32.13 Firewalls And Internet Access

Mechanisms that control *internet access* handle the problem of screening a particular network or an organization from unwanted communication. Such mechanisms can help prevent outsiders from: obtaining information, changing information, or disrupting communication on an organization's intranet. Successful access control requires a careful combination of restrictions on network topology, intermediate information staging, and packet filters.

A single technique known as an *internet firewall*[†], has emerged as the basis for internet access control. An organization places a firewall at its connection to external networks (e.g., the global Internet). A firewall partitions an internet into two regions, referred to informally as the *inside* and *outside*.

32.14 Multiple Connections And Weakest Links

Although concept seems simple, details complicate firewall construction. First, an organization's intranet can have multiple external connections. The organization must form a *security perimeter* by installing a firewall at each external connection. To guarantee that the perimeter is effective, all firewalls must be configured to use exactly the same access restrictions. Otherwise, it may be possible to circumvent the restrictions imposed by one firewall by entering the organization's internet through another[‡].

We can summarize:

An organization that has multiple external connections must install a firewall on each external connection and must coordinate all firewalls. Failure to restrict access identically on all firewalls can leave the organization vulnerable.

[†]The term *firewall* is derived from building architecture in which a firewall is a thick, fireproof partition that makes a section of a building impenetrable to fire.

[‡]The well-known idea that security is only as strong as the weakest point has been termed the *weakest link axiom* in reference to the adage that a chain is only as strong as its weakest link.

32.15 Firewall Implementation

How should a firewall be implemented? In theory, a firewall simply blocks all unauthorized communication between computers in the organization and computers outside the organization. In practice, the details depend on the network technology, the capacity of the connection, the traffic load, and the organization's policies. Thus, no single solution works for all organizations; building an effective, customized firewall can be difficult.

To operate at network speeds, a firewall must have hardware and software optimized for the task. Fortunately, most commercial routers include a high-speed filtering mechanism that can be used to perform much of the necessary work. A manager can configure the filter in a router to request that the router block specified datagrams. As we discuss the details of filter mechanisms, we will see how filters form the basic building blocks of a firewall. Later we will see how filters can be used in conjunction with another mechanism to provide communication that is safe, but flexible.

32.16 Packet-Level Filters

Many commercial routers offer a mechanism that augments normal routing and permits a manager to further control packet processing. Informally called a *packet filter*, the mechanism requires the manager to specify how the router should dispose of each datagram. For example, the manager might choose to *filter* (i.e. block) all datagrams that come from a particular source or those used by a particular application, while choosing to route other datagrams to their destination.

The term *packet filter* arises because the filtering mechanism does not keep a record of interaction or a history of previous datagrams. Instead, the filter considers each datagram separately. When a datagram first arrives, the router passes the datagram through its packet filter before performing any other processing. If the filter rejects the datagram, the router drops it immediately.

Because TCP/IP does not dictate a standard for packet filters, each router vendor is free to choose the capabilities of their packet filter as well as the interface a manager uses to configure the filter. Some routers permit a manager to configure separate filter actions for each interface, while others have a single configuration for all interfaces. Usually, when specifying datagrams that the filter should block, a manager can list any combination of source IP address, destination IP address, protocol, source protocol port number, and destination protocol port number. For example, Figure 32.6 illustrates a filter specification.

In the example, the manager has chosen to block incoming datagrams destined for a few well-known services and to block one case of outgoing datagrams. The filter blocks all outgoing datagrams that originate from any host address matching the 16-bit prefix of *128.5.0.0* that are destined for a remote e-mail server (TCP port 25). The filter also blocks incoming datagrams destined for FTP (TCP port 21), TELNET (TCP port 23), WHOIS (UDP port 43), TFTP (UDP port 69), or FINGER (TCP port 79).

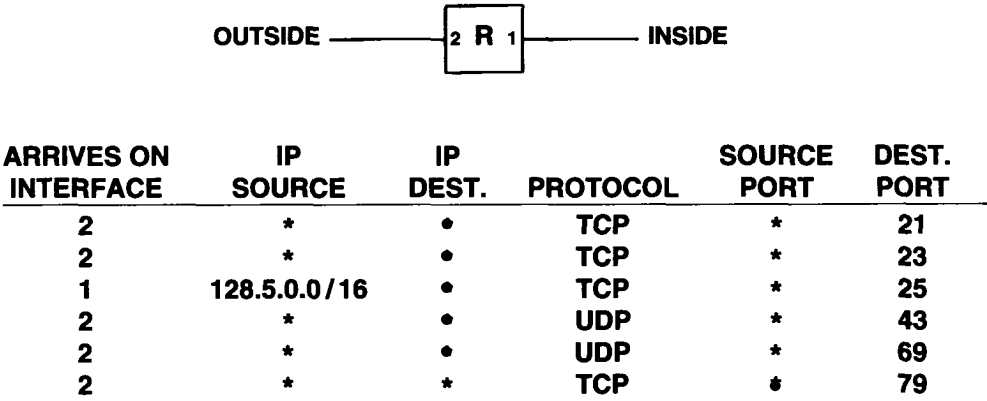


Figure 32.6 A router with two interfaces and an example datagram filter specification. A router that includes a packet filter forms the basic building block of a firewall.

32.17 Security And Packet Filter Specification

Although the example filter configuration in Figure 32.6 specifies a small list of services that should be blocked, such an approach does not work well for an effective firewall. There are three reasons. First, the number of well-known ports is large and growing rapidly. Thus, listing each service requires a manager to update the list continually; an error of omission can leave the firewall vulnerable. Second, much of the traffic on an internet does not travel to or from a well-known port. In addition to programmers who can choose port numbers for their private client-server applications, services like *Remote Procedure Call (RPC)* assign ports dynamically. Third, listing ports of well-known services leaves the firewall vulnerable to *tunneling*. Tunneling can circumvent security if a host or router on the inside agrees to accept encapsulated datagrams from an outsider, remove one layer of encapsulation, and forward the datagram on to the service that would otherwise be restricted by the firewall.

How can a firewall use a packet filter effectively? The answer lies in reversing the idea of a filter: instead of specifying the datagrams that should be filtered, a firewall should be configured to block all datagrams except those destined for specific networks, hosts, and protocol ports for which external communication has been approved. Thus, a manager begins with the assumption that communication is not allowed, and then must examine the organization's information policy carefully before enabling any port. In fact, many packet filters allow a manager to specify a set of datagrams to admit instead of a set of datagrams to block. We can summarize:

To be effective, a firewall that uses datagram filtering should restrict access to all IP sources, IP destinations, protocols, and protocol ports except those computers, networks, and services the organization explicitly decides to make available externally. A packet filter that allows a manager to specify which datagrams to admit instead of which datagrams to block can make such restrictions easy to specify.

32.18 The Consequence Of Restricted Access For Clients

A blanket prohibition on datagrams arriving for an unknown protocol port seems to solve many potential security problems by preventing outsiders from accessing arbitrary servers in the organization. Such a firewall has an interesting consequence: it also prevents an arbitrary computer inside the firewall from becoming a client that accesses a service outside the firewall. To understand why, recall that although each server operates at a well-known port, a client does not. When a client program begins execution, it requests the operating system to select a protocol port number that is neither among the well-known ports nor currently in use on the client's computer. When it attempts to communicate with a server outside the organization, a client will generate one or more datagrams and send them to the server. Each outgoing datagram has the client's protocol port as the source port and the server's well-known protocol port as the destination port. The firewall will not block such datagrams as they leave. When it generates a response, the server reverses the protocol ports. The client's port becomes the destination port and the server's port becomes the source port. When the datagram carrying the response reaches the firewall, however, it will be blocked because the destination port is not approved. Thus, we can see an important idea:

If an organization's firewall restricts incoming datagrams except for ports that correspond to services the organization makes available externally, an arbitrary application inside the organization cannot become a client of a server outside the organization.

32.19 Proxy Access Through A Firewall

Of course, not all organizations configure their firewalls to block all datagrams destined for unknown protocol ports. In cases where a secure firewall is needed to prevent unwanted access, however, users on the inside need a safe mechanism that provides access to services outside. That mechanism forms the second major piece of firewall architecture.

In general, an organization can only provide safe access to outside services through a secure computer. Instead of trying to make all computer systems in the organization secure (a daunting task), an organization usually associates one secure computer with

each firewall, and installs a set of application gateways on that computer. Because the computer must be strongly fortified to serve as a secure communication channel, it is often called a *bastion host*. Figure 32.7 illustrates the concept.

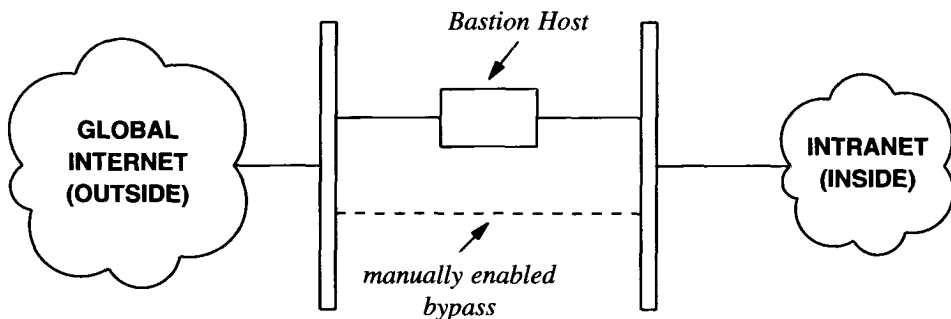


Figure 32.7 The conceptual organization of a bastion host embedded in a firewall. The bastion host provides secure access to outside services without requiring an organization to admit datagrams with arbitrary destinations.

As the figure shows, the firewall has two conceptual barriers. The outer barrier blocks all incoming traffic except (1) datagrams destined for services on the bastion host that the organization chooses to make available externally, and (2) datagrams destined for clients on the bastion host. The inner barrier blocks incoming traffic except datagrams that originate on the bastion host. Most firewalls also include a *manual bypass* that enables managers to temporarily pass some or all traffic between a host inside the organization and a host outside (e.g., for testing or debugging the network).

To understand how a bastion host operates, consider Web access. Because the firewall prevents the user's computer from receiving incoming datagrams, the user cannot use a browser for direct access. Instead, the organization arranges a proxy server on the bastion host. Inside the organization, each browser is configured to use the proxy. Whenever a user selects a link or enters a URL, their browser contacts the proxy. The proxy contacts the server, obtains the specified page, and then delivers it internally.

32.20 The Details Of Firewall Architecture

Now that we understand the basic firewall concept, the implementation should appear straightforward. Conceptually, each of the barriers shown in Figure 32.7 requires a router that has a packet filter†. Networks interconnect the routers and a bastion host. For example, an organization that connects to the global Internet might choose to implement a firewall as Figure 32.8 shows.

†Some organizations use a *one-armed firewall* configuration in which a single physical router implements all the functionality.

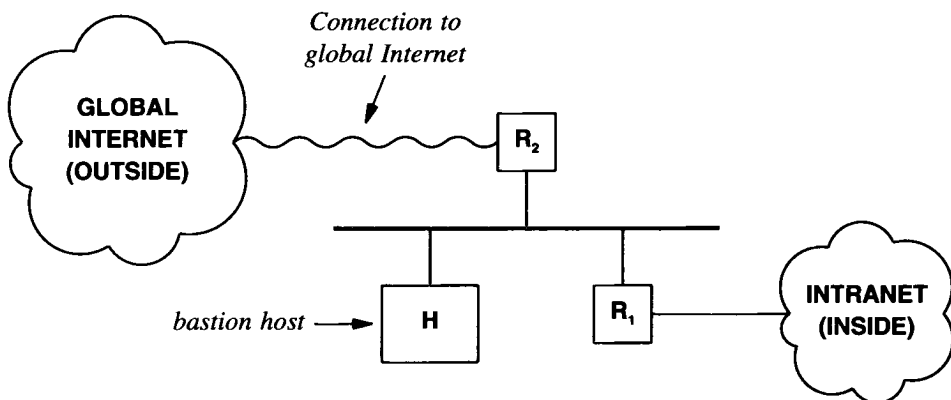


Figure 32.8 A firewall implemented with two routers and a bastion host. One of the routers has a connection to the rest of the Internet.

As the figure shows, router R_2 implements the outer barrier; it filters all traffic except datagrams destined for the bastion host, H . Router R_1 implements the inner barrier that isolates the rest of the corporate intranet from outsiders; it blocks all incoming datagrams except those that originate on the bastion host.

Of course, the safety of an entire firewall depends on the safety of the bastion host. If an intruder can gain access to the computer system running on the bastion host, they will gain access to the entire inside internet. Moreover, an intruder can exploit security flaws in either the operating system on the bastion host or the network applications it runs. Thus, managers must be particularly careful when choosing and configuring software for a bastion host. In summary:

Although a bastion host is essential for communication through a firewall, the security of the firewall depends on the safety of the bastion host. An intruder who exploits a security flaw in the bastion host operating system can gain access to hosts inside the firewall.

32.21 Stub Network

It may seem that Figure 32.8 contains a superfluous network that connects the two routers and the bastion host. Such a network is often called a *stub network* because it is small (i.e., stubby). The question arises, “Is the stub network necessary or could a site place the bastion host on one of its production networks?” The answer depends on the traffic expected from the outside. The stub network isolates the organization from incoming datagram traffic. In particular, because router R_2 admits all datagrams destined for the bastion host, an outsider can send an arbitrary number of such datagrams across

the stub network. If an external connection is slow relative to the capacity of a stub network, a separate physical wire may be unnecessary. However, a stub network is usually an inexpensive way for an organization to protect itself against disruption of service on an internal production network.

32.22 An Alternative Firewall Implementation

The firewall implementation in Figure 32.8 works well for an organization that has a single serial connection to the rest of the global Internet. Some sites have a different interconnection topology. For example, suppose a company has three or four large customers who each need to deposit or extract large volumes of information. The company wishes to have a single firewall, but allow connections to multiple sites†. Figure 32.9 illustrates one possible firewall architecture that accommodates multiple external connections.

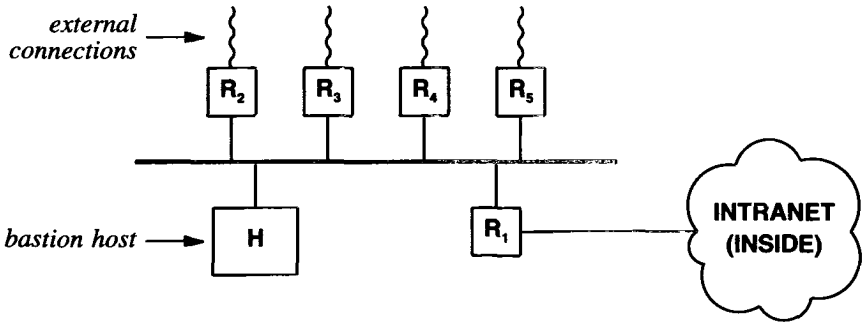


Figure 32.9 An alternative firewall architecture that permits multiple external connections through a single firewall. Using one firewall for multiple connections can reduce the cost.

As the figure shows, the alternative architecture extends a firewall by providing an outer network at which external connections terminate. Router R_1 acts as in Figure 32.8 to protect the site by restricting incoming datagrams to those sent from the bastion host. Routers R_2 through R_5 each connect one external site to the firewall.

To understand why firewalls with multiple connections often use a router per connection, recall that all sites mistrust one another. That is, the organization running the firewall does not trust any of the external organizations completely, and none of the external organizations trust one another completely. The packet filter in a router on a given external connection can be configured to restrict traffic on that particular connection. As a result, the owner of the firewall can guarantee that although all external connections share a single, common network, no datagram from one external connection will pass to another. Thus, the organization running the firewall can assure customers that it is safe to connect. To summarize:

†A single firewall can be less expensive and easier to administrate than a separate firewall per connection.

When multiple external sites connect through a single firewall, an architecture that has a router per external connection can prevent unwanted packet flow from one external site to another.

32.23 Monitoring And Logging

Monitoring is one of the most important aspects of a firewall design. The network manager responsible for a firewall needs to be aware of attempts to bypass security. Unless a firewall reports incidents, a manager may be unaware of problems.

Monitoring can be *active* or *passive*. In active monitoring, a firewall notifies a manager whenever an incident occurs. The chief advantage of active monitoring is speed — a manager finds out about a potential problem immediately. The chief disadvantage is that active monitors often produce so much information that a manager cannot comprehend it or notice problems. Thus, most managers prefer passive monitoring, or a combination of passive monitoring with a few high-risk incidents also reported by an active monitor.

In passive monitoring, a firewall logs a record of each incident in a file on disk. A passive monitor usually records information about normal traffic (e.g., simple statistics) as well as datagrams that are filtered. A manager can access the log at any time; most managers use a computer program. The chief advantage of passive monitoring arises from its record of events — a manager can consult the log to observe trends and when a security problem does occur, review the history of events that led to the problem. More important, a manager can analyze the log periodically (e.g., daily) to determine whether attempts to access the organization increase or decrease over time.

32.24 Summary

Security problems arise because an internet can connect organizations that do not have mutual trust. Several technologies are available to help ensure that information remains secure when being sent across an internet. IPsec allows a user to choose between two basic schemes: one that provides authentication of the datagram and one that provides authentication plus privacy. IPsec modifies a datagram either by inserting an Authentication Header or by using an Encapsulating Security Payload, which inserts a header and trailer and encrypts the data being sent. IPsec provides a general framework that allows each pair of communicating entities to choose an encryption algorithm. Because security is often used with tunneling (e.g., in a VPN), IPsec defines a secure tunnel mode.

The firewall mechanism is used to control internet access. An organization places a firewall at each external connection to guarantee that the organization's intranet remains free from unauthorized traffic. A firewall consists of two barriers and a secure computer called a bastion host. Each barrier uses a packet filter to restrict datagram traffic. The bastion host offers externally-visible servers, and runs proxy servers that al-

low users to access outside servers. The filters are configured according to the organization's information policy. Usually, the firewall blocks all datagrams arriving from external sources except those datagrams destined for the bastion host.

A firewall can be implemented in one of several ways; the choice depends on details such as the number of external connections. In many cases, each barrier in a firewall is implemented with a router that contains a packet filter. A firewall can also use a stub network to keep external traffic off an organization's production networks.

FOR FURTHER STUDY

In the mid 1990s, the IETF announced a major emphasis on security, and required each working group to consider the security implications of its designs. Consequently, many RFCs address issues of internet security and propose policies, procedures, and mechanisms. Kent and Atkinson [RFC 2401] defines the IPsec architecture. Kent and Atkinson [RFC 2402] specifies the IPsec authentication header, and [RFC 2406] specifies the encapsulating security payload.

Many RFCs describe security for particular application protocols. For example, Wijnen et. al. [RFC 2575] presents the view-based security and Blumenthal and Wijnen [RFC 2574] presents a user-based security model, both are intended for use with SNMPv3.

Cheswick and Bellovin [1994] discusses firewalls and other topics related to the secure operation of TCP/IP internets. Kohl and Neuman [RFC 1510] describes the *kerberos* authentication service, and Borman [RFC 1411] discusses how *kerberos* can be used to authenticate TELNET.

EXERCISES

- 32.1 Many sites that use a bastion host arrange for software to scan all incoming files before admitting them to the organization. Why do organizations scan files?
- 32.2 Read the description of a packet filter for a commercially available router. What features does it offer?
- 32.3 Collect a log of all traffic entering your site. Analyze the log to determine the percentage of traffic that arrives from or is destined to a well-known protocol port. Do the results surprise you?
- 32.4 If encryption software is available on your computer, measure the time required to encrypt a 10 Mbyte file, transfer it to another computer, and decrypt it. Compare the result to the time required for the transfer if no encryption is used.
- 32.5 Survey users at your site to determine if they send sensitive information in e-mail. Are users aware that SMTP transfers messages in ASCII, and that anyone watching network traffic can see the contents of an e-mail message?

- 32.6** Survey employees at your site to find out how many use modems and personal computers to import or export information. Ask if they understand the organization's information policy.
- 32.7** Can a firewall be used with other protocol suites such as AppleTalk or Netware? Why or why not?
- 32.8** Can a firewall be combined with NAT? What are the consequences?
- 32.9** The military only releases information to those who "need to know." Will such a scheme work for all information in your organization? Why or why not?
- 32.10** Give two reasons why the group of people who administer an organization's security policies should be separate from the group of people who administer the organization's computer and network systems.
- 32.11** Some organizations use firewalls to isolate groups of users internally. Give examples of ways that internal firewalls can improve network performance and examples of ways internal firewalls can degrade network performance.
- 32.12** If your organization uses IPsec, find out which algorithms are being used. What is the key size?

The Future Of TCP/IP (IPv6)

33.1 Introduction

Evolution of TCP/IP technology is intertwined with evolution of the global Internet for several reasons. First, the Internet is the largest installed TCP/IP internet, so many problems related to scale arise in the Internet before they surface in other TCP/IP internets. Second, funding for TCP/IP research and engineering comes from companies and government agencies that use the operational Internet, so they tend to fund projects that impact the Internet. Third, because most researchers use the global Internet daily, they have immediate motivation to solve problems that will improve service and extend functionality.

With millions of users at tens of thousands of sites around the world depending on the global Internet as part of their daily work environment, it might appear that the Internet is a completely stable production facility. We have passed the early stage of development in which every user was also an expert, and entered a stage in which few users understand the technology. Despite appearances, however, neither the Internet nor the TCP/IP protocol suite is static. Groups discover new ways to use the technology. Researchers solve new networking problems, and engineers improve the underlying mechanisms. In short, the technology continues to evolve.

The purpose of this chapter is to consider the ongoing evolutionary process and examine one of the most significant engineering efforts: a proposed revision of IP. When the proposal is adopted by vendors, it will have a major impact on TCP/IP and the global Internet.

33.2 Why Change?

The basic TCP/IP technology has worked well for over two decades. Why should it change? In a broad sense, the motivation revising the protocols arises from changes in underlying technologies and uses.

- *New Computer And Communication Technologies.* Computer and network hardware continues to evolve. As new technologies emerge, they are incorporated into the Internet.
- *New Applications.* As programmers invent new ways to use TCP/IP, additional protocol support is needed. For example, the emphasis on IP telephony has led to investigations of protocols for real-time data delivery.
- *Increases In Size And Load.* The global Internet has experienced many years of sustained exponential growth, doubling in size every nine months or faster. In 1999, on the average, a new host appeared on the Internet every two seconds. Traffic has also increased rapidly as animated graphics and video proliferate.

33.3 New Policies

As it expands into new countries, the Internet changes in a fundamental way: it gains new administrative authorities. Changes in authority produce changes in administrative policies, and mandate new mechanisms to enforce those policies. As we have seen, both the architecture of the connected Internet and the protocols it uses are evolving away from a centralized core model. Evolution continues as more national backbone networks attach, producing increasingly complex policies regulating interaction. When multiple corporations interconnect private TCP/IP internets, they face similar problems as they try to define policies for interaction and then develop mechanisms to enforce those policies. Thus, many of the research and engineering efforts surrounding TCP/IP continue to focus on finding ways to accommodate new administrative groups.

33.4 Motivation For Changing IPv4

Version 4 of the Internet Protocol (*IPv4*) provides the basic communication mechanism of the TCP/IP suite and the global Internet; it has remained almost unchanged since its inception in the late 1970s†. The longevity of version 4 shows that the design is flexible and powerful. Since the time IPv4 was designed, processor performance has increased over two orders of magnitude, typical memory sizes have increased by over a factor of 100, network bandwidth of the Internet backbone has risen by a factor of 7000, LAN technologies have emerged, and the number of hosts on the

†Versions 1 through 3 were never formally assigned, and version number 5 was assigned to the *ST* protocol.

Internet has risen from a handful to over 56 million. Furthermore, because the changes did not occur simultaneously, adapting to them has been a continual process.

Despite its sound design, IPv4 must be replaced soon. Chapter 10 describes the main motivation for updating IP: the imminent address space limitations. When IP was designed, a 32-bit address space was more than sufficient. Only a handful of organizations used a LAN; fewer had a corporate WAN. Now, however, most medium-sized corporations have multiple LANs, and most large corporations have a corporate WAN. Consequently, even with careful assignment and NAT technology, the current 32-bit IP address space cannot accommodate projected growth of the global Internet beyond the year 2020.

Although the need for a larger address space is the most immediate motivation, other factors contributed to the new design. In particular, to make IP better suited to real-time applications, thought was given to supporting systems that associate a datagram with a preassigned resource reservation. To make electronic commerce safer, the next version of IP is designed to include support for security features such as authentication.

33.5 The Road To A New Version Of IP

It took many years for the IETF to formulate a new version of IP. Because the IETF produces *open* standards, it invited the entire community to participate in the process. Computer manufacturers, hardware and software vendors, users, managers, programmers, telephone companies, and the cable television industry all specified their requirements for the next version of IP, and all commented on specific proposals.

Many designs were proposed to serve a particular purpose or a particular community. One of the major proposals would have made IP more sophisticated at the cost of increased complexity and processing overhead. Another design proposed using a modification of the OSI CLNS protocol. A third major design proposed retaining most of the ideas in IP, but making simple extensions to accommodate larger addresses. The design, known as *SIP*[†] (*Simple IP*), became the basis for an extended proposal that included ideas from other proposals. The extended version was named *Simple IP Plus* (*SIPP*), and eventually emerged as the design selected as a basis for the next IP.

Choosing a new version of IP was not easy. The popularity of the Internet means that the market for IP products around the world is staggering. Many groups see the economic opportunity, and hope that the new version of IP will help them gain an edge over the competition. In addition, personalities have been involved — some individuals hold strong technical opinions; others see active participation as a path to a promotion. Consequently, the discussions generated heated arguments.

[†]The acronym *SIP* now refers to the *Session Initiation Protocol* which is used for signaling (e.g., for IP telephony).

33.6 The Name Of The Next IP

The IETF decided to assign the revision of IP version number 6 and to name it *IPv6*[†] to distinguish it from the current *IPv4*. The choice to skip version number 5 arose after a series of mistakes and misunderstandings. In one mistake, the IAB caused widespread confusion by inadvertently publishing a policy statement that referred to the next version of IP as *IP version 7*. In a misunderstanding, an experimental protocol known as the *Stream Protocol (ST)* was assigned version number 5. The assignment led some to conclude that ST had been selected as the replacement for IP. In the end, the IETF chose 6 because doing so eliminated confusion.

33.7 Features Of IPv6

The proposed IPv6 protocol retains many of the features that contributed to the success of IPv4. In fact, the designers have characterized IPv6 as being basically the same as IPv4 with a few modifications. For example, IPv6 still supports connectionless delivery (i.e., each datagram is routed independently), allows the sender to choose the size of a datagram, and requires the sender to specify the maximum number of hops a datagram can make before being terminated. As we will see, IPv6 also retains most of the concepts provided by IPv4 options, including facilities for fragmentation and source routing.

Despite many conceptual similarities, IPv6 changes most of the protocol details. For example, IPv6 uses larger addresses, and adds a few new features. More important, IPv6 completely revises the datagram format by replacing IPv4's variable-length options field by a series of fixed-format headers. We will examine details after considering major changes and the underlying motivation for each.

The changes introduced by IPv6 can be grouped into seven categories:

- *Larger Addresses.* The new address size is the most noticeable change. IPv6 quadruples the size of an IPv4 address from 32 bits to 128 bits. The IPv6 address space is so large that it cannot be exhausted in the foreseeable future.
- *Extended Address Hierarchy.* IPv6 uses the larger address space to create additional levels of addressing hierarchy. In particular, IPv6 can define a hierarchy of ISPs as well as a hierarchical structure within a given site.
- *Flexible Header Format.* IPv6 uses an entirely new and incompatible datagram format. Unlike the IPv4 fixed-format header, IPv6 defines a set of optional headers.
- *Improved Options.* Like IPv4, IPv6 allows a datagram to include optional control information. IPv6 includes new options that provide additional facilities not available in IPv4.

[†]Some documents refer to the effort as “IP — The Next Generation,” *IPng*.

- *Provision For Protocol Extension.* Perhaps the most significant change in IPv6 is a move away from a protocol that fully specifies all details to a protocol that can permit additional features. The extension capability has the potential to allow the IETF to adapt the protocol to changes in underlying network hardware or to new applications.
- *Support For Autoconfiguration And Renumbering.* IPv6 provides facilities that allow computers on an isolated network to assign themselves addresses and begin communicating without depending on a router or manual configuration. The protocol also includes a facility that permits a manager to renumber networks dynamically.
- *Support For Resource Allocation.* IPv6 has two facilities that permit preallocation of network resources: a flow abstraction and a differentiated service specification. The latter will use the same approach as IPv4's differentiated services.

33.8 General Form Of An IPv6 Datagram

IPv6 completely changes the datagram format. As Figure 33.1 shows, an IPv6 datagram has a fixed-size *base header* followed by zero or more *extension headers*, followed by data.

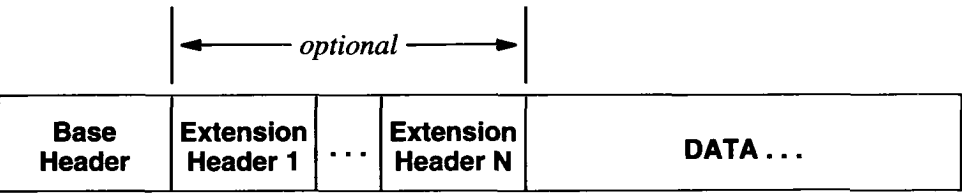


Figure 33.1 The general form of an IPv6 datagram with multiple headers. Only the base header is required; extension headers are optional.

33.9 IPv6 Base Header Format

Interestingly, although it must accommodate larger addresses, an IPv6 base header contains less information than an IPv4 datagram header. Options and some of the fixed fields that appear in an IPv4 datagram header have been moved to extension headers in IPv6. In general, the changes in the datagram header reflect changes in the protocol:

- Alignment has been changed from 32-bit to 64-bit multiples.

- The header length field has been eliminated, and the datagram length field has been replaced by a *PAYLOAD LENGTH* field.
- The size of source and destination address fields has been increased to 16 octets each.
- Fragmentation information has been moved out of fixed fields in the base header into an extension header.
- The *TIME-TO-LIVE* field has been replaced by a *HOP LIMIT* field.
- The *SERVICE TYPE* is renamed to be a *TRAFFIC CLASS* field, and extended with a *FLOW LABEL* field.
- The *PROTOCOL* field has been replaced by a field that specifies the type of the next header.

Figure 33.2 shows the contents and format of an IPv6 base header. Several fields in an IPv6 base header correspond directly to fields in an IPv4 header. As in IPv4, the initial 4-bit *VERS* field specifies the version of the protocol; *VERS* always contains 6 in an IPv6 datagram. As in IPv4, the *SOURCE ADDRESS* and *DESTINATION ADDRESS* fields specify the addresses of the sender and intended recipient. In IPv6, however, each address requires 16 octets. The *HOP LIMIT* field corresponds to the IPv4 *TIME-TO-LIVE* field. Unlike IPv4, which interprets a time-to-live as a combination of hop-count and maximum time, IPv6 interprets the value as giving a strict bound on the maximum number of hops a datagram can make before being discarded.

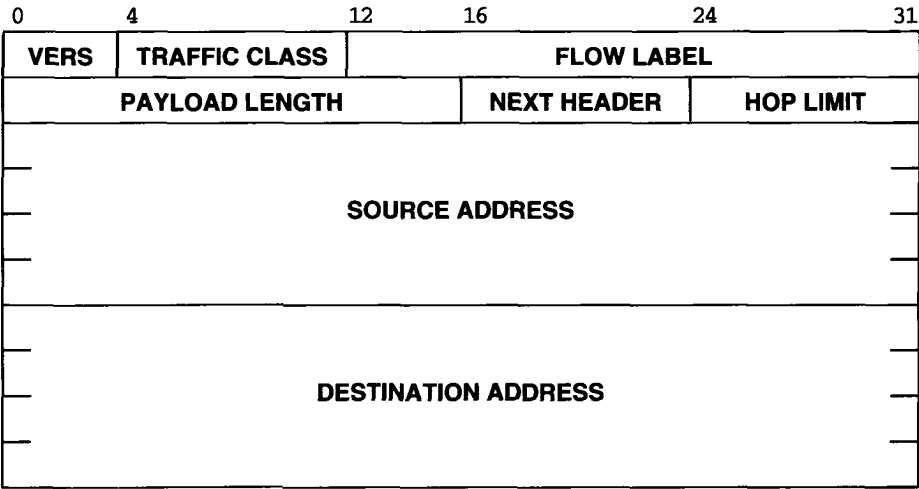


Figure 33.2 The format of the 40-octet IPv6 base header. Each IPv6 datagram begins with a base header.

IPv6 handles datagram length specifications in a new way. First, because the size of the base header is fixed at 40 octets, the base header does not include a field for the header length. Second, IPv6 replaces IPv4's datagram length field by a 16-bit *PAYLOAD LENGTH* field that specifies the number of octets carried in the datagram excluding the header itself. Thus, an IPv6 datagram can contain 64K octets of data.

Two fields in the base header are used in making forwarding decisions. The IPv4 *SERVICE CLASS* field has been renamed *TRAFFIC CLASS*. In addition, a new mechanism in IPv6 supports resource reservation and allows a router to associate each datagram with a given resource allocation. The underlying abstraction, a *flow*, consists of a path through an internet along which intermediate routers guarantee a specific quality of service. Field *FLOW LABEL* in the base header contains information that routers use to associate a datagram with a specific flow and priority. For example, two applications that need to send video can establish a flow on which the delay and bandwidth is guaranteed. Alternatively, a network provider may require a subscriber to specify the quality of service desired, and then use a flow to limit the traffic a specific computer or a specific application sends. Note that flows can also be used within a given organization to manage network resources and ensure that all applications receive a fair share. A router uses the combination of datagram source address and flow identifier when associating a datagram with a specific flow. To summarize:

Each IPv6 datagram begins with a 40-octet base header that includes fields for the source and destination addresses, the maximum hop limit, the traffic class, the flow label, and the type of the next header. Thus, an IPv6 datagram must contain at least 40 octets in addition to the data.

33.10 IPv6 Extension Headers

The paradigm of a fixed base header followed by a set of optional extension headers was chosen as a compromise between generality and efficiency. To be totally general, IPv6 needs to include mechanisms to support functions such as fragmentation, source routing, and authentication. However, choosing to allocate fixed fields in the datagram header for all mechanisms is inefficient because most datagrams do not use all mechanisms; the large IPv6 address size exacerbates the inefficiency. For example, when sending a datagram across a single local area network, a header that contains empty address fields can occupy a substantial fraction of each frame. More important, the designers realize that no one can predict which facilities will be needed.

The IPv6 extension header paradigm works similar to IPv4 options — a sender can choose which extension headers to include in a given datagram and which to omit. Thus, extension headers provide maximum flexibility. We can summarize:

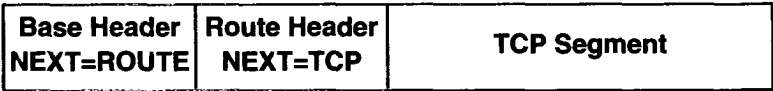
IPv6 extension headers are similar to IPv4 options. Each datagram includes extension headers for only those facilities that the datagram uses.

33.11 Parsing An IPv6 Datagram

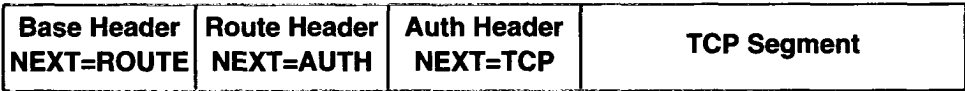
Each of the base and extension headers contains a *NEXT HEADER* field. Software on intermediate routers and at the final destination that process a datagram use the values in the *NEXT HEADER* fields to parse the datagram. Extracting all header information from an IPv6 datagram requires a sequential search through the headers. For example, Figure 33.3 shows the *NEXT HEADER* fields of three datagrams that contain zero, one, and two extension headers.



(a)



(b)



(c)

Figure 33.3 Three datagrams with (a) only a base header, (b) a base header and one extension, and (c) a base header plus two extensions. The *NEXT HEADER* field in each header specifies the type of the following header.

Of course, parsing an IPv6 datagram that only has a base header and data is as efficient as parsing an IPv4 datagram. Furthermore, intermediate routers only need to examine the *hop-by-hop* extension header; only endpoints process other extension headers.

33.12 IPv6 Fragmentation And Reassembly

As in IPv4, IPv6 arranges for the ultimate destination to perform datagram reassembly. However, the designers chose to make changes that avoid fragmentation by routers. Recall that IPv4 requires an intermediate router to fragment any datagram that is too large for the MTU of the network over which it must travel. In IPv6, fragmentation is end-to-end; no fragmentation needs to occur in intermediate routers. The source, which is responsible for fragmentation, has two choices: it can either use the *guaranteed minimum MTU* of 1280 octets or perform *Path MTU Discovery* to identify the minimum MTU along the path to the destination. In either case, the source fragments the datagram so that each fragment is less than the expected path MTU.

The IPv6 base header does not contain fields analogous to the fields used for fragmentation in an IPv4 header. Instead, when fragmentation is needed, the source inserts a small extension header after the base header in each fragment. Figure 33.4 shows the contents of a *Fragment Extension Header*.

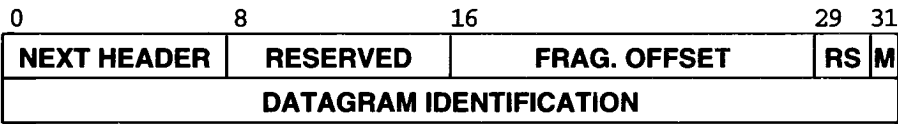


Figure 33.4 The format of a Fragment Extension Header.

IPv6 retains the basic IPv4 fragmentation functionality. Each fragment must be a multiple of 8 octets, the single bit in the *M* field marks the last fragment like the IPv4 *MORE FRAGMENTS* bit, and the *DATAGRAM IDENTIFICATION* field carries a unique ID that the receiver uses to group fragments†. Finally, field *RS* is currently reserved; the two bits are set to zero on transmission and ignored by the receiver.

33.13 The Consequence Of End-To-End Fragmentation

The motivation for using end-to-end fragmentation lies in its ability to reduce overhead in routers and permit each router to handle more datagrams per unit time. Indeed, the CPU overhead required for IPv4 fragmentation can be significant — in a conventional router, the CPU can reach 100% utilization if the router fragments many or all of the datagrams it receives. However, end-to-end fragmentation has an important consequence: it alters the fundamental IPv4 assumption that routes change dynamically.

To understand the consequence of end-to-end fragmentation, recall that IPv4 is designed to permit routes to change at any time. For example, if a network or router fails, traffic can be routed along a different path. The chief advantage of such a system is flexibility — traffic can be routed along an alternate path without disrupting service and without informing the source or destination. In IPv6, however, routes cannot be

†IPv6 expands the IPv4 *IDENTIFICATION* field to 32 bits to accommodate higher speed networks.

changed as easily because a change in a route can also change the path MTU. If the path MTU along a new route is less than the path MTU along the original route, either an intermediate router must fragment the datagram or the original source must be informed. The problem can be summarized:

An internet protocol that uses end-to-end fragmentation requires a sender to discover the path MTU to each destination, and to fragment any outgoing datagram that is larger than the path MTU. End-to-end fragmentation does not accommodate route changes.

To solve the problem of route changes that affect the path MTU, IPv6 includes a new ICMP error message. When a router discovers that fragmentation is needed, it sends the message back to the source. When it receives such a message, the source performs another path MTU discovery to determine the new minimum MTU, and then fragments datagrams according to the new value.

33.14 IPv6 Source Routing

IPv6 retains the ability for a sender to specify a loose source route. Unlike IPv4, in which source routing is provided by options, IPv6 uses a separate extension header. As Figure 33.5 shows, the first four fields of the Routing Header are fixed. Field *ROUTING TYPE* specifies the type of routing information; the only type that has been defined, type 0, corresponds to loose source routing. The *TYPE-SPECIFIC DATA* field contains a list of addresses of routers through which the datagram must pass. Field *SEG LEFT* specifies the total number of addresses that remain in the list. Finally field *HDR EXT LEN* specifies the size of the Routing Header.

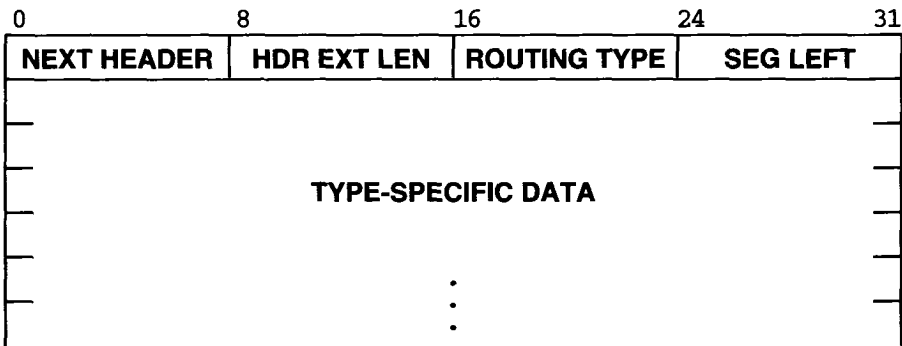


Figure 33.5 The format of an IPv6 Routing Header. Only type 0 (loose source route) is currently defined.

33.15 IPv6 Options

It may seem that IPv6 extension headers completely replace IPv4 options. However, the designers propose two additional extension headers to accommodate miscellaneous information not included in other extension headers. The additional headers are a *Hop By Hop Extension Header* and an *End To End Extension Header*. As the names imply, the two headers separate the set of options that should be examined at each hop from the set that are only interpreted at the destination.

Although each of the two option headers has a unique type code, both headers use the format illustrated in Figure 33.6.

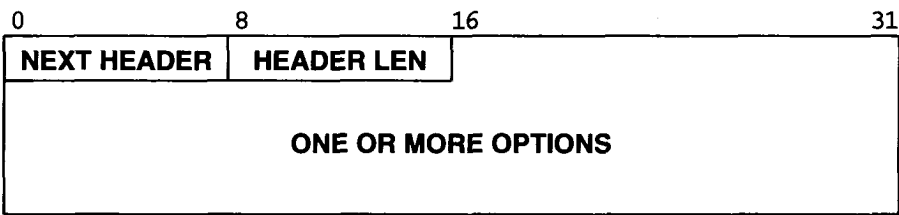


Figure 33.6 The format of an IPv6 option extension header. Both the *hop-by-hop* and *end-to-end* option headers use the same format; the *NEXT HEADER* field of the previous header distinguishes between the two types.

As usual, field *NEXT HEADER* gives the type of the header that follows. Because an option header does not have fixed size, the field labeled *HEADER LEN* specifies the total length of the header. The area labeled *ONE OR MORE OPTIONS* represents a sequence of individual options. Figure 33.7 illustrates how each individual option is encoded with a type, length, and value†; options are not aligned or padded.

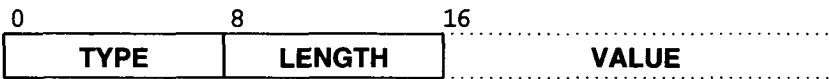


Figure 33.7 Encoding of an individual option in an IPv6 option extension header. Each option consists of a one-octet type and a one-octet length followed by zero or more octets of data for the option.

As the figure shows, IPv6 options follow the same form as IPv4 options. Each option begins with a one-octet *TYPE* field followed by a one-octet *LENGTH* field. If the option requires additional data, octets that comprise the *VALUE* follow the *LENGTH*.

†In the literature, an encoding of type, length, and value is sometimes called a *TLV encoding*.

The two high-order bits of each option *TYPE* field specify how a host or router should dispose of the datagram if it does not understand the option:

Bits In Type	Meaning
00	Skip this option
01	Discard datagram; do not send ICMP message
10	Discard datagram; send ICMP message to source
11	Discard datagram; send ICMP for non-multicast

In addition, the third bit in the *TYPE* field specifies whether the option can change in transit. Having such information is important for authentication — the contents of an option that can change in transit are treated as zeroes for purposes of authentication.

33.16 Size Of The IPv6 Address Space

In IPv6, each address occupies 16 octets, four times the size of an IPv4 address. The large address space guarantees that IPv6 can tolerate any reasonable address assignment scheme. In fact, if the designers decide to change the addressing scheme later, the address space is sufficiently large to accommodate a reassignment.

It is difficult to comprehend the size of the IPv6 address space. One way to look at it relates the magnitude to the size of the population: the address space is so large that every person on the planet can have sufficient addresses to have their own internet as large as the current Internet. A second way to think of IPv6 addressing relates it to the physical space available: the earth’s surface has approximately 5.1×10^8 square kilometers, meaning that there are over 10^{24} addresses per square meter of the earth’s surface. Another way to understand the size relates it to address exhaustion. For example, consider how long it would take to assign all possible addresses. A 16-octet integer can hold 2^{128} values. Thus, the address space is greater than 3.4×10^{38} . If addresses are assigned at the rate of one million addresses every microsecond, it would take over 10^{20} years to assign all possible addresses.

33.17 IPv6 Colon Hexadecimal Notation

Although it solves the problem of having insufficient capacity, the large address size poses an interesting new problem: humans who maintain internets must read, enter, and manipulate such addresses. Obviously, binary notation is untenable. However, the dotted decimal notation used for IPv4 does not make such addresses sufficiently compact either. To understand why, consider an example 128-bit number expressed in dotted decimal notation:

104.230.140.100.255.255.255.255.0.0.17.128.150.10.255.255

To help make address slightly more compact and easier to enter, the IPv6 designers propose using *colon hexadecimal notation* (abbreviated *colon hex*) in which the value of each 16-bit quantity is represented in hexadecimal separated by colons. For example, when the value shown above in dotted decimal notation has been translated to colon hex notation and printed using the same spacing, it becomes:

68E6:8C64:FFFF:FFFF:0:1180:96A:FFFF

Colon hex notation has the obvious advantage of requiring fewer digits and fewer separator characters than dotted decimal. In addition, colon hex notation includes two techniques that make it extremely useful. First, colon hex notation allows *zero compression* in which a string of repeated zeros is replaced by a pair of colons. For example, the address:

FF05:0:0:0:0:0:0:B3

can be written:

FF05::B3

To ensure that zero compression produces an unambiguous interpretation, the proposal specifies that it can be applied only once in any address. Zero compression is especially useful when used with the proposed address assignment scheme because many addresses will contain contiguous strings of zeros. Second, colon hex notation incorporates dotted decimal suffixes; we will see that such combinations are intended to be used during the transition from IPv4 to IPv6. For example, the following string is valid colon hex notation:

0:0:0:0:0:0:128.10.2.1

Note that although the numbers separated by colons each specify the value of a 16-bit quantity, numbers in the dotted decimal portion each specify the value of one octet. Of course, zero compression can be used with the number above to produce an equivalent colon hex string that looks quite similar to an IPv4 address:

::128.10.2.1

Finally, IPv6 extends CIDR-like notation by allowing an address to be followed by a slash and an integer that specifies a number of bits. For example,

12AB::CD30:0:0:0:0 / 60

specifies the first 60 bits of the address or 12AB00000000CD3 in hexadecimal.

33.18 Three Basic IPv6 Address Types

Like IPv4, IPv6 associates an address with a specific network connection, not with a specific computer. Thus, address assignments are similar to IPv4: an IPv6 router has two or more addresses, and an IPv6 host with one network connection needs only one address. IPv6 also retains (and extends) the IPv4 address hierarchy in which a physical network is assigned a prefix. However, to make address assignment and modification easier, IPv6 permits multiple prefixes to be assigned to a given network, and allows a computer to have multiple, simultaneous addresses assigned to a given interface.

In addition to permitting multiple, simultaneous addresses per network connection, IPv6 expands, and in some cases unifies, IPv4 special addresses. In general, a destination address on a datagram falls into one of three categories:

- Unicast The destination address specifies a single computer (host or router); the datagram should be routed to the destination along a shortest path.
- Anycast The destination is a set of computers, possibly at different locations, that all share a single address; the datagram should be routed along a shortest path and delivered to exactly one member of the group (i.e., the closest member)†.
- Multicast The destination is a set of computers, possibly at multiple locations. One copy of the datagram will be delivered to each member of the group using hardware multicast or broadcast if viable.

33.19 The Duality Of Broadcast And Multicast

IPv6 does not use the terms *broadcast* or *directed broadcast* to refer to delivery to all computers on a physical network or to a logical IP subnet. Instead, it uses the term *multicast*, and treats broadcast as a special form of multicast. The choice may seem odd to anyone who understands network hardware because more hardware technologies support broadcast than support multicast. In fact, a hardware engineer is likely to view multicasting as a restricted form of broadcasting — the hardware sends a multicast packet to all computers on the network exactly like a broadcast packet, and the interface hardware on each computer filters all multicast packets except those that software has instructed the interface hardware to accept.

In theory, the choice between multicast and limited forms of broadcast is irrelevant because one can be simulated with the other. That is, broadcasting and multicasting are duals of one another that provide the same functionality. To understand why, consider how to simulate one with the other. If broadcast is available, a packet can be delivered to a group by sending it to all machines and arranging for software on each computer to decide whether to accept or discard the incoming packet. If multicast is available, a

†Anycast addresses were formerly known as *cluster* addresses.

packet can be delivered to all machines by arranging for all machines to listen to one multicast group similar to the *all hosts* group discussed in Chapter 17.

33.20 An Engineering Choice And Simulated Broadcast

Knowing that broadcasting and multicasting are theoretical duals of one another does not help choose between them. To see why the designers of IPv6 chose multicasting as the central abstraction instead of broadcasting, consider applications instead of looking at the underlying hardware. An application either needs to communicate with a single application or with a group of applications. Direct communication is handled best via unicast; group communication is handled best by multicast or broadcast. To provide the most flexibility, group membership should not be determined by network connections, because group members can reside at arbitrary locations. Using broadcast for all group communication does not scale to handle an internet as large as the global Internet.

Not surprisingly, the designers pre-define some multicast addresses that can be used in place of an IPv4 network broadcast address. Thus, in addition to its own unicast address, each router is required to accept packets addressed to the *All Routers* multicast groups for its local environment.

33.21 Proposed IPv6 Address Space Assignment

The question of how to partition the IPv6 address space has generated much discussion. There are two central issues: how to manage address assignment and how to map an address to a route. The first issue focuses on the practical problem of devising a hierarchy of authority. Unlike the current Internet, which uses a two-level hierarchy of network prefix (assigned by the Internet authority) and host suffix (assigned by the organization), the large address space in IPv6 permits a multi-level hierarchy or multiple hierarchies. The second issue focuses on computational efficiency. Independent of the hierarchy of authority that assigns addresses, a router must examine each datagram and choose a path to the destination. To keep the cost of high-speed routers low, the processing time required to choose a path must be kept small.

As Figure 33.8 shows, the designers of IPv6 propose assigning address classes in a way similar to the scheme used for IPv4. Although the first 8 bits of an address are sufficient to identify its type, the address space is not partitioned into sections of equal size.

Binary Prefix	Type Of Address	Part Of Address Space
0000 0000	Reserved (IPv4 compatibility)	1/256
0000 0001	Unassigned	1/256
0000 001	NSAP Addresses	1/128
0000 010	IPX Addresses	1/128
0000 011	Unassigned	1/128
0000 1	Unassigned	1/32
0001	Unassigned	1/16
001	Aggregatable Global Unicast	1/8
010	Unassigned	1/8
011	Unassigned	1/8
100	Unassigned	1/8
101	Unassigned	1/8
110	Unassigned	1/8
1110	Unassigned	1/16
1111 0	Unassigned	1/32
1111 10	Unassigned	1/64
1111 110	Unassigned	1/128
1111 1110 0	Unassigned	1/512
1111 1110 10	Link-Local Unicast Addresses	1/1024
1111 1110 11	Site-Local Unicast Addresses	1/1024
1111 1111	Multicast Addresses	1/256

Figure 33.8 The proposed division of IPv6 addresses into types, which are analogous to IPv4 classes. As in IPv4, the prefix of an address determines its address type.

As the figure shows, only 15% of the address space has been assigned at present. The IETF will use the remaining portions as demand grows. Despite the sparse assignment, addresses have been chosen to make processing more efficient. For example, the high-order octet of an address distinguishes between multicast (all 1 bits) and unicast (a mixture of 0's and 1's).

33.22 Embedded IPv4 Addresses And Transition

Although the prefix 0000 0000 is labeled *Reserved* in the figure, the designers plan to use a small fraction of addresses in that section to encode IPv4 addresses. In particular, any address that begins with 80 zero bits followed by 16 bits of all ones or 16 bits of all zeros contains an IPv4 address in the low-order 32 bits. The value of the 16-bit field indicates whether the node also has a conventional IPv6 unicast address. Figure 33.9 illustrates the two forms.

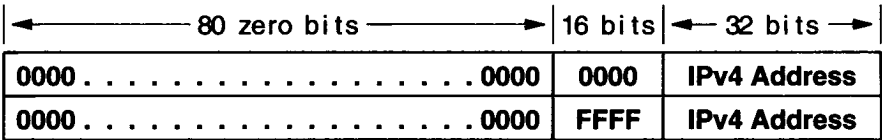


Figure 33.9 The encoding of an IPv4 address in an IPv6 address. The 16-bit field contains *0000* if the node also has a conventional IPv6 address, and *FFFF* if it does not.

The encoding will be needed during a transition from IPv4 to IPv6 for two reasons. First, a computer may choose to upgrade from IPv4 to IPv6 software before it has been assigned a valid IPv6 address. Second, a computer running IPv6 software may need to communicate with a computer that runs only IPv4 software.

Having a way to encode an IPv4 address in an IPv6 address does not solve the problem of making the two version interoperate. In addition to address encoding, translation is needed. To use a translator, an IPv6 computer generates a datagram that contains the IPv6 encoding of the IPv4 destination address. The IPv6 computer sends the datagram to a translator, which uses IPv4 to communicate with the destination. When the translator receives a reply from the destination, it translates the IPv4 datagram to IPv6 and sends it back to the IPv6 source.

It may seem that translating protocol addresses could fail because higher layer protocols verify address integrity. In particular, TCP and UDP, use a *pseudo header* in their checksum computation. The pseudo header includes both the source and destination protocol addresses, so changing such addresses could affect the computation. However, the designers planned carefully to allow TCP or UDP on an IPv4 machine to communicate with the corresponding transport protocol on an IPv6 machine. To avoid checksum mismatch, the IPv6 encoding of an IPv4 address has been chosen so that the 16-bit 1's complement checksum for both an IPv4 address and the IPv6 encoding of the address are identical. The point is:

In addition to choosing technical details of a new Internet Protocol, the IETF work on IPv6 has focused on finding a way to transition from the current protocol to the new protocol. In particular, the current proposal for IPv6 allows one to encode an IPv4 address inside an IPv6 address such that address translation does not change the pseudo header checksum.

33.23 Unspecified And Loopback Addresses

As in IPv4, a few IPv6 addresses have been assigned special meaning. For example, the all 0's address:

0:0:0:0:0:0:0

is an *unspecified address* which cannot be assigned to any computer or used as a destination. It is only used as a source address during bootstrap by a computer that has not yet learned its address.

Like IPv4, IPv6 also has a *loopback address* that is used for testing software. The IPv6 loopback address is:

0:0:0:0:0:0:0:1

Any datagram sent to the loopback address will be delivered to the local machine; it must never be used as a destination address on an outgoing datagram.

33.24 Unicast Address Hierarchy

One of the most important changes between IPv4 and IPv6 arises from the allocation strategy used for unicast addresses and the resulting address hierarchy. Recall that the original IPv4 addressing scheme used a two-level hierarchy in which an address is divided into a globally unique prefix and a suffix. IPv6 extends the concept by adopting an address hierarchy with three conceptual levels as Figure 33.10 illustrates.

Level	Purpose
1	Globally-known public topology
2	Individual site
3	Individual network interface

Figure 33.10 The three conceptual levels of the IPv6 unicast address hierarchy. In practice, an address has additional structure.

The two lowest levels of the conceptual hierarchy are easiest to understand because they correspond to identifiable entities. The lowest level corresponds to a single attachment between a computer and a network. The middle level of the hierarchy corresponds to a set of computers and networks located at a *site*, which implies both contiguous physical connectivity and a single organization that owns and operates the equipment. We will see that the addressing scheme accommodates both large and small sites, and allows a site to have complex internal structure.

To provide flexibility, the top level of the hierarchy, which is labeled *public topology*, is not precisely defined. In general, one can think of the public topology as a “section” of the global Internet that is available for public access. Two types of public topology are envisioned. The first type corresponds to a major *Internet Service Provider (ISP)* that provides long-haul service to customers, which are known as *subscribers*. The second type, which is called an *exchange*, is a newly envisioned organization. According to the designers, exchanges will provide two functions. First, an exchange will operate like a NAP to interconnect major ISPs and pass traffic among them. Second, unlike current NAPs, exchanges will also service individual subscribers, which means that the exchange will assign the subscriber an address. The chief advantage of an address assigned by an exchange is that the address will not specify an ISP. Thus, a subscriber will be free to move from one ISP to another.

33.25 Aggregatable Global Unicast Address Structure

Authority for IPv6 address assignment flows down the hierarchy. Each top-level organization (e.g., an ISP or exchange) is assigned a unique prefix. When an organization becomes a subscriber of a top-level ISP, the organization is assigned a unique number for its site. Finally, a manager must assign a number to each network connection. To make routing efficient, successive sets of bits in the address are reserved for each assignment. Figure 33.11 illustrates the format, which is known as a *aggregatable global unicast address* format.

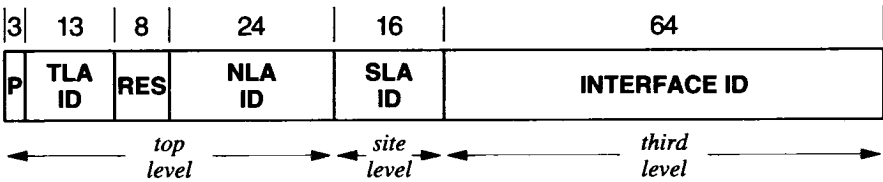


Figure 33.11 The division of an IPv6 aggregatable global unicast address into separate fields along with an indication of how those fields correspond to the three-level hierarchy.

The 3-bit field labeled *P* in the figure corresponds to the *format prefix*, which is 001 for an aggregatable global unicast address. The 8-bit *RES* field is reserved for the future and contains zeroes. Remaining fields in the address are arranged to make routing efficient. In particular, fields that correspond to the highest level of the hierarchy are grouped together to comprise the most significant bits of the address. Field *TLA ID* contains an identifier used for *Top-Level Aggregation* (i.e., a unique identifier assigned to the ISP or exchange that owns the address). The owner of the address uses field *NLA* to provide *Next-Level Aggregation* (e.g., to identify a particular subscriber).

The 16-bit field labeled *SLA ID (Site-Level Aggregation)* is available for a specific site to use. The designers envision it being used much like an IPv4 subnet field. Thus, a site with only a few networks can choose to treat the field as a network identifier, and a site that has many networks can use the field to partition networks into groups which can then be arranged in a hierarchy. To create a one-level hierarchy at the site, the organization must use a prefix to identify the group and a suffix to identify a particular network in the group. As with IPv4 subnetting, the division into groups improves routing efficiency because a routing table only contains routes to each of the other groups rather than to each individual network.

33.26 Interface Identifiers

As Figure 33.11 shows, the low-order 64 bits of an IPv6 aggregatable unicast address identifies a specific network interface. Unlike IPv4, however, the IPv6 suffix was chosen to be large enough to accommodate a direct encoding of the interface hardware address. Encoding a hardware address in an IP address has two consequences. First, IPv6 does not use ARP to resolve an IP address to a hardware address. Instead, IPv6 uses a *neighbor discovery protocol* available with a new version of ICMP (*ICMPv6*) to allow a node to determine which computers are its directly connected neighbors. Second, to guarantee interoperability, all computers must use the same encoding for a hardware address. Consequently, the IPv6 standards specify exactly how to encode various forms of hardware address. In the simplest case, the hardware address is placed directly in the IPv6 address; some formats use more complex transformations.

Two example encodings will help clarify the concept. For example, IEEE defines a standard 64-bit globally unique address format known as *EUI-64*. The only change needed when encoding an EUI-64 address in an IPv6 address consists of inverting bit 6 in the high-order octet of the address, which indicates whether the address is known to be globally unique.

A more complex change is required for a conventional 48-bit Ethernet address. Figure 33.12 illustrates the encoding. As the figure shows, bits from the original address are not contiguous in the encoded form. Instead, 16 bits with hexadecimal value 0xFFFE are inserted in the middle. In addition, bit 6, which indicates whether the address has global scope, is changed from 0 to 1. Remaining bits of the address, including the group bit (labeled *g*), the ID of the company that manufactured the interface (labeled *c*), and the manufacturer's extension are copied as shown.

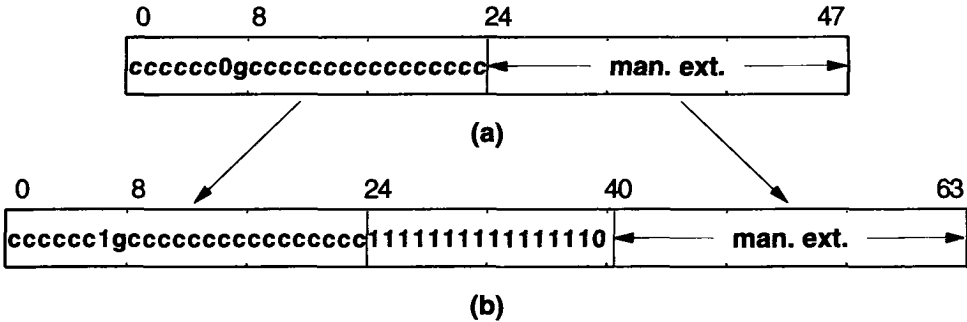


Figure 33.12 (a) The format of a 48-bit IEEE 802 address used with Ethernet, with bits labeled *c* specifying the company that manufactured the interface and bits in the *man. ext.* field specifying an extension the manufacturer chose to uniquely identify the unit, and (b) the encoding of the address in the low order 64 bits of an IPv6 unicast address.

33.27 Additional Hierarchy

Although the unicast address format in Figure 33.11 implies a strict hierarchy, many additional levels are possible. For example, bits of the *NLA ID* can be used to create a hierarchy of providers. Similarly, the 16-bit *SLA ID* can be divided to create a hierarchy within an organization. The large number of bits provides more flexibility than IPv4 subnetting. An organization can choose to divide into a two-level hierarchy of areas and assign subnets within each area. Alternatively, an organization can choose a three-level hierarchy of areas, subareas, and subnets within each subarea.

33.28 Local Addresses

In addition to the global unicast addresses described above, IPv6 includes prefixes for unicast addresses that have local scope. As Figure 33.8 shows, the standard defines two types: a *link-local address* is restricted to a single network, and a *site-local address* is restricted to a single site. Routers honor the scoping rules; they do not forward datagrams containing locally-scoped addresses outside the specified scope.

Local addresses solve two problems. Link-local addresses provide communication across a single physical network without danger of the datagram being forwarded across the internet. For example, when it performs neighbor discovery, an IPv6 node uses a link-local address. The scope rules specify that only computers on the same physical network as the sender will receive neighbor discovery messages. Similarly, computers connected to an *isolated network* (i.e., a network that does not have routers attached) can use link-local addresses to communicate.

Unlike a datagram containing link-local addresses, routers can forward datagrams containing site-local addresses throughout an entire organization. However, routers are prohibited from forwarding such datagrams to the global Internet. Thus, site-local addresses correspond to what IPv4 calls *private* or *nonroutable* addresses. An organization can assign and use site-local addresses throughout its private intranet without obtaining and assigning globally unique prefixes.

33.29 Autoconfiguration And Renumbering

IPv6 is designed to support *serverless autoconfiguration*[†] that allows computers to communicate without requiring a manager to specify an address. Two facilities discussed above make autoconfiguration possible and efficient: link-local addressing and embedded interface identifiers. To begin, a computer generates a link-local address by combining the link-local prefix:

1111 1110 10

with 54 zero bits and its 64-bit interface identifier.

Once it verifies that the link-local address is unique, a computer uses the address to send a *router solicitation* that requests additional information from a router. If a router is present on the network, the router responds by sending a *router advertisement* to inform the computer about prefixes that can be used for site-local or global addresses. When a router advertisement arrives, the computer makes the sender its default router. Finally, a flag in the advertisement tells the computer whether to rely on autoconfiguration or to use a conventional *managed configuration* (i.e., DHCP).

To facilitate network *renumbering*, IPv6 allows routers to limit the time a computer can retain a prefix. To do so, a router advertisement specifies two time values for each prefix: a valid lifetime and a preferred lifetime. A host must listen for additional router advertisements. When the preferred lifetime of a prefix expires, the prefix remains valid, but the host must use another prefix for all communication when possible. When the valid lifetime expires, the host must stop using the prefix, even if existing communication is in progress.

33.30 Summary

The IETF has defined a next generation of the Internet Protocol which is known as IPv6 because it has been assigned version number 6. IPv6 retains many of the basic concepts from the current protocol, IPv4, but changes most details. Like IPv4, IPv6 provides a connectionless, best-effort datagram delivery service. However, the IPv6 datagram format differs from the IPv4 format, and IPv6 provides new features such as authentication and support for flow-labeling.

IPv6 organizes each datagram as a series of headers followed by data. A datagram always begins with a 40-octet base header, which contains source and destination ad-

[†]Serverless autoconfiguration is also called *stateless autoconfiguration*.

addresses, a traffic class, and a flow identifier. The base header may be followed by zero or more extension headers, followed by data. Extension headers are optional — IPv6 uses them to hold much of the information IPv4 encodes in options.

An IPv6 address is 128 bits long, making the address space so large that the space cannot be exhausted in the foreseeable future. IPv6 uses address prefixes to determine the location and interpretation of remaining address fields. In addition to traditional unicast and multicast addresses, IPv6 also defines anycast addresses. A single anycast address can be assigned to a set of computers; a datagram sent to the address is delivered to exactly one computer in the set (i.e., the computer closest to the source).

IPv6 supports autoconfiguration and renumbering. Each host on an isolated network generates a unique link-local address which it uses for communication. The host also uses the link-local address to discover routers and obtain site-local and global prefix information. To facilitate renumbering, all prefixes are assigned a lifetime; a host must use a new prefix if the lifetime on an existing prefix expires.

FOR FURTHER STUDY

Many RFCs have appeared that contain information pertinent to IPv6. Deering and Hinden [RFC 2460] specifies the basic protocol. Thomson and Narten [RFC 2462] describes stateless address autoconfiguration. Narten, Nordmark, and Simpson [RFC 2461] discusses neighbor discovery. Conta and Deering [RFC 2463] specifies ICMPv6 as a companion to IPv6. Crawford [RFC 2464] describes encapsulation of IPv6 for transmission over Ethernet networks.

Many RFCs focus on IPv6 addressing. Hinden and Deering [RFC 2373] describes the basic addressing architecture including the meanings of prefixes. Hinden, O'Dell, and Deering [RFC 2374] considers the aggregatable global unicast address format. Hinden and Deering [RFC 2375] specifies multicast address assignments. Johnson and Deering [RFC 2526] describes reserved anycast addresses. Information about the 64-bit EUI format can be found in:

<http://standards.ieee.org/regauth/oui/tutorials/EUI64.html>

EXERCISES

- 33.1 The current standard for IPv6 has no header checksum. What are the advantages and disadvantages of this approach?
- 33.2 How should extension headers be ordered to minimize processing time?
- 33.3 Although IPv6 addresses are assigned hierarchically, a router does not need to parse an address completely to select a route. Devise an algorithm and data structure for efficient routing. (Hint: consider a longest-match approach.)

- 33.4** Argue that 128-bit addresses are larger than needed, and that 96 bits provides sufficient capacity.
- 33.5** Assume your organization intends to adopt IPv6. Devise an address scheme the organization will use to assign each host an address. Did you choose a hierarchical assignment within your organization? Why or why not?
- 33.6** What is the chief advantage of encoding an Ethernet address in an IPv6 address? The chief disadvantage?
- 33.7** Consider a host that forms a link-local address by encoding its 48-bit Ethernet address with the standard link-local prefix. Is the resulting address guaranteed to be unique on the network? Why or why not?
- 33.8** In the previous exercise, does the standard specify that the host must use the Neighbor Discovery Protocol to verify that the address is unique? Why or why not?
- 33.9** If you were asked to choose sizes for the top-level, next-level, and site ID fields of an IPv6 unicast address, how large would you make each? Why?
- 33.10** Read about the IPv6 authentication and security headers. Why are two headers proposed?
- 33.11** How does the IPv6 minimum MTU of 1280 affect its flexibility?

Appendix 1

A Guide To RFCs

Introduction

Most of the written information about TCP/IP and the connected Internet, including its architecture, protocols, and history, can be found in a series of reports known as *Request For Comments* or *RFCs*. An informal, loosely coordinated set of notes, RFCs are unusually rich in information and color. Before we consider the more serious aspects of RFCs, it is fitting that we take a few minutes to pay attention to the colorful side. A good place to begin is with Cerf's poem *'Twas the Night Before Start-up* (RFC 968), a humorous parody that describes some of the problems encountered when starting a new network. Knowing not to take itself too seriously has pervaded the Internet effort. Anyone who can remember both their first Internet meeting, filled with networking jargon, and Lewis Carroll's *Jabberwocky*, filled with strangely twisted English, will know exactly why D. L. Covill put them together in *ARPAWOCKY* (RFC 527).

Other RFCs seem equally frivolous. Interspersed amid the descriptions of ideas that would turn out to dramatically change networking, we find notes like RFC 416, written in early November, 1972: *The ARC System will be Unavailable for Use During Thanksgiving Week*. It says exactly what you think it says. Or consider Crispin's tongue-in-cheek humor found in RFC 748, which describes the *TELNET Randomly-Lose Option* (a proposed option for TELNET that makes it randomly drop characters). In fact, any RFC dated April 1 should be considered a joke. If such items do not seem insignificant, think about the seventy-five RFCs listed as *never issued*. All were assigned a number and had an author, but none ever saw the light of day. The holes in the numbering scheme remain, preserved as little reminders of ideas that vaporized or work that remains incomplete.

Even after the silly, lighthearted, and useless RFCs have been removed, the remaining documents do not conform to most standards for scientific writing. Unlike scholarly scientific journals that concentrate on identifying papers of important archival interest, screening them carefully, and filing them for posterity, RFCs provide a record of ongoing conversations among the principals involved in designing, building, measuring, and using the global Internet. The reader understands at once that RFCs include the thoughts of researchers on the leading edge of technological innovation, not the studied opinions of scholars who have completely mastered a subject. The authors are not always sure of the consequences of their proposals, or even of the contents, but they clearly realize the issues are too complex to understand without community discussion. For example, RFC 1173 purports to document the “oral traditions” (which is an oxymoron because it became part of the written tradition once the RFC was published).

Despite the inconsistencies in RFCs that sometimes make them difficult for beginners to understand, the RFC mechanism has evolved and now works extremely well. Because RFCs are available electronically, information is propagated to the community quickly. Because they span a broad range of interests, practitioners as well as designers contribute. Because they record informal conversations, RFCs capture discussions and not merely final conclusions. Even the disagreements and contradictory proposals are useful in showing what the designers considered before settling on a given protocol (and readers interested in the history of a particular idea or protocol can use RFCs to follow it from its inception to its current state).

Importance Of Host And Gateway Requirements Documents

Unlike most RFCs, which concentrate on a single idea or protocol, three special RFCs cover a broad range of protocols. The special documents are entitled *Requirements for Internet Routers* and *Requirements for Internet Hosts* (parts 1 and 2).

The requirements documents, published after many years of experience with the TCP/IP protocols, are considered a major revision to the protocol standards. In essence, requirement documents each review many protocols. They point out known weaknesses or ambiguities in the RFCs that define the protocols, state conventions that have been adopted by vendors, document problems that occur in practice, and list solutions to those problems that have been accumulated through experience. The RFCs for individual protocols have *not* been updated to include changes and updates from the requirements documents. Thus, readers must be careful to always consult the requirements documents when studying a particular protocol.

RFC Numerology

RFCs cover a surprisingly large range of sizes, with the average size being 47504.5 bytes. The largest, RFC 1166 (Internet numbers), contains 566778 bytes, while the smallest consists of a 27-byte text file:

This RFC was never issued.

A few interesting coincidences have occurred. For example, the ASCII text file for RFC 41 contains exactly 41 lines of text, and the ASCII text file for RFC 854, exactly 854 lines. RFC 1996 has a number that matches the year in which it was published. However, the number for no other RFC will match the year of publication.

The quantity of RFCs published per year varies widely. Figure A1.1 illustrates how the rate has changed over time. The surge of work in the 1970s represents an initial period of building; the high rate of publication in the 1990s has resulted from commercialization.

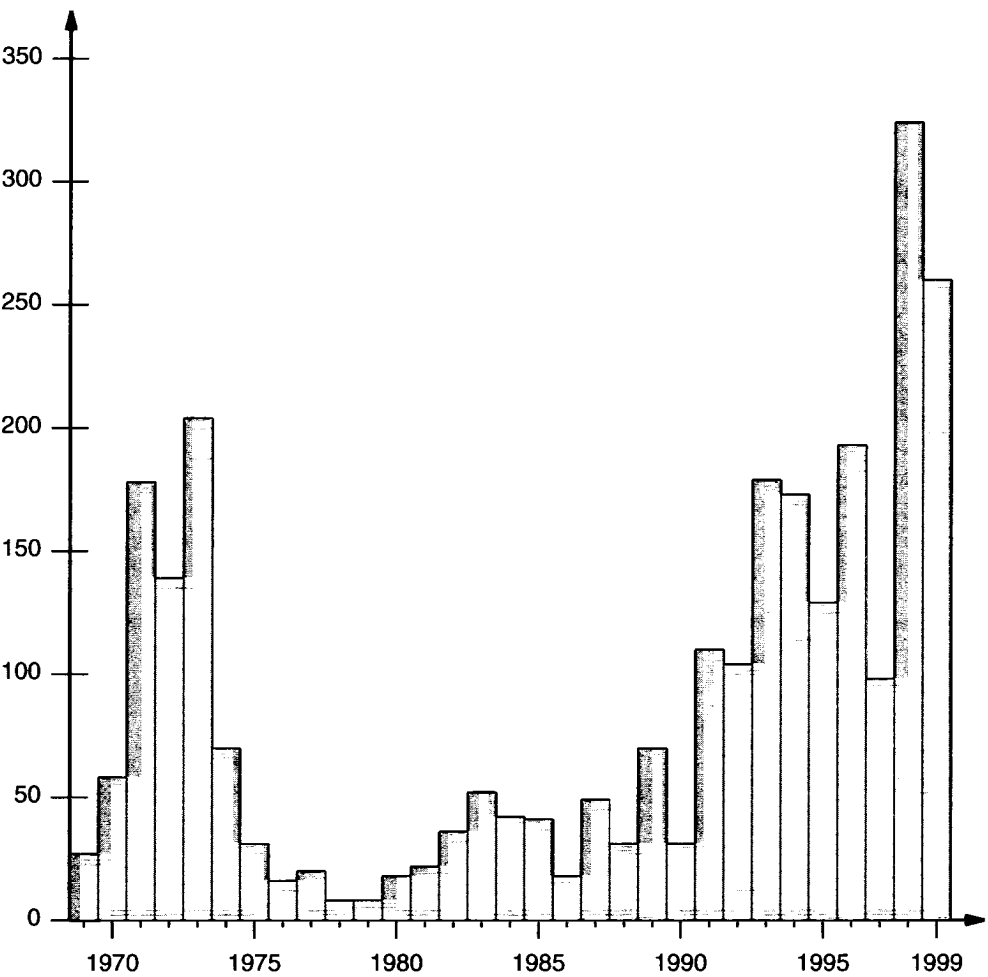


Figure A1.1 The number of RFCs published per year.

How To Obtain An RFC Over The Internet

RFCs are available electronically from many repositories around the world. Check with your local network administrator to find the site nearest you or begin with the following URL:

<http://www.rfc-editor.org>

Browsing Through RFCs

There are several indexes that can help one browse through RFCs. ISI publishes an index of all RFCs in chronological order. Readers often need to know which RFC contains the latest version of an official Internet protocol or which protocols are official and which are unofficial. To accommodate such needs, the IAB periodically publishes an RFC entitled *INTERNET OFFICIAL PROTOCOL STANDARDS*, which provides a list of all protocols that have been adopted as TCP/IP standards, along with the number of the most recent RFC or RFCs describing each protocol. RFC 1602, *The Internet Standards Process – Revision 2*, describes the Internet standardization process and defines the meaning of the terms *proposed standard*, *draft standard*, *Internet standard*, *required*, *recommended*, and *historic*.

Despite the available indexes, browsing through RFCs can be difficult, especially when the reader is searching for information pertinent to a given topic, which may be spread across RFCs published in many years. Browsing is particularly difficult because titles do not provide sufficient identification of the content. (How could one guess from the title *Leaving Well Enough Alone* that the RFC pertains to FTP?) Finally, having multiple RFCs with a single title (e.g., Internet Numbers) can be confusing because the reader cannot easily tell whether a document is out-of-date without checking the archive.

RFCs Arranged By Topic

The final section of this appendix provides help in finding information in RFCs because it contains a list of the first 2728 RFCs arranged by topic. Readers can find an earlier topical index in RFC 1000, which also includes an annotated chronological listing of the first 1000 RFCs. Although long, RFC 1000 is highly recommended as a source of authoritative and valuable critique – its introduction is especially fascinating. Recalling the origin of RFCs along with the origin of the ARPANET, the introduction captures the spirit of adventure and energy that still characterizes the Internet.

RFCs Organized By Major Category And Subtopic

1. Administrative

1a. Assigned Internet Numbers (official values used by protocols)

1700, 1340, 1166, 1117, 1062, 1060, 1020, 1010, 997, 990, 960, 943, 923, 900, 870, 820, 790, 776, 770, 762, 758, 755, 750, 739, 717, 604, 503, 433, 349, 322, 317, 204, 179, 175, 167

1b. Official IAB Standards and Other Lists of Protocols

2500, 2400, 2300, 2200, 2000, 1920, 1880, 1800, 1780, 1720, 1610, 1600, 1540, 1500, 1410, 1360, 1280, 1250, 1200, 1140, 1130, 1100, 1083, 1011, 991, 961, 944, 924, 901, 880, 840, 694, 661, 617, 582, 580, 552
774, 766

1c. Meeting Notes and Minutes

2316 –Report of the IAB Security Architecture Workshop
2130 –The Report of the IAB Character Set Workshop held 29 February - 1 March, 1996
1862 –Report of the IAB Workshop on Internet Information Infrastructure, October 12-14, 1994
1636 –Report of IAB Workshop on Security in the Internet Architecture - February 8-10, 1994
1588 –White Pages Meeting Report
1210 –Network and infrastructure user requirements for transatlantic research collaboration: Brussels, July 16-18, and Washington July 24-25, 1990
1152 –Workshop report: Internet research steering group workshop on very-high-speed networks
1077 –Critical issues in high bandwidth networking
1019 –Report of the Workshop on Environments for Computational Mathematics
1017 –Network requirements for scientific research: Internet task force on scientific computing
910, 807 - Multimedia mail meeting notes
898 – Gateway special interest group meeting notes
808, 805, 469 - Summary of computer mail services meeting held at BBN on 10 January 1979
585 – ARPANET users interest working group meeting
549, 396, 282, 253 - Minutes of Network Graphics Group meeting, 15-17 July 1973
371 – Demonstration at International Computer Communications Conference
327 – Data and File Transfer workshop notes
316 – ARPA Network Data Management Working Group
164, 131, 108, 101, 82, 77, 63, 37, 21 - Minutes of Network Working Group meeting, 5/16 through 5/19/71

1d. Meeting Announcements and Group Overviews

1160, 1120 - Internet Activities Board
828 – Data communications: IFIP's international "network" of experts
631 – International meeting on minicomputers and data communication: Call for papers
584 – Charter for ARPANET Users Interest Working Group
537 – Announcement of NGG meeting July 16-17
526 – Technical meeting: Digital image processing software systems
504 – Distributed resources workshop announcement
483 – Cancellation of the resource notebook framework meeting

- 474, 314, 246, 232, 134 - Announcement of NGWG meeting: Call for papers
- 471 – Workshop on multi-site executive programs
- 461 – Telnet Protocol meeting announcement
- 457 – TIPUG
- 456 – Memorandum: Date change of mail meeting
- 454 – File Transfer Protocol - meeting announcement and a new proposed document
- 453 – Meeting announcement to discuss a network mail system
- 374 – IMP System Announcement
- 359 – Status of the Release of the New IMP System (2600)
- 343, 331 - IMP System change notification
- 324 – RJE Protocol meeting
- 323 – Formation of Network Measurement Group (NMG)
- 320 – Workshop on Hard Copy Line Printers
- 309 – Data and File Transfer Workshop Announcement
- 299 – Information Management System
- 295 – Report of the Protocol Workshop, 12 October 1971
- 291, 188, 173 - Data Management Meeting Announcement
- 245, 234, 207, 140, 116, 99, 87, 85, 75, 43, 35 - Reservations for Network Group meeting
- 222 – Subject: System programmer's workshop
- 212 – NWG meeting on network usage
- 157 – Invitation to the Second Symposium on Problems in the Optimization of Data Communications Systems
- 149 – Best Laid Plans
- 130 – Response to RFC 111: Pressure from the chairman
- 111 – Pressure from the Chairman
- 48 – Possible protocol plateau
- 46 – ARPA Network protocol notes

1e. Distribution Lists

- 402, 363, 329, 303, 300, 211, 168, 155 - ARPA Network Mailing Lists
- 69 – Distribution List Change for MIT
- 52 – Updated distribution list

1f. Policies Documents

- 2717 –Registration Procedures for URL Scheme Names
- 2506 –Media Feature Tag Registration Procedure
- 2489 –Procedure for Defining New DHCP Options
- 2418, 1603 - IETF Working Group Guidelines and Procedures
- 2282, 2027 - IAB and IESG Selection, Confirmation, and Recall Process: Operation of the Nominating and Recall Committees
- 2278 –IANA Charset Registration Procedures
- 2277 –IETF Policy on Character Sets and Languages
- 2146, 1816, 1811 - US Government Internet Domain Names
- 2135 –Internet Society By-Laws
- 2050 –Internet Registry IP Allocation Guidelines
- 2042 –Registering New BGP Attribute Types
- 2014 –IRTF Research Group Guidelines and Procedures
- 1956 –Registration in the MIL Domain
- 1930 –Guidelines for creation, selection, and registration of an Autonomous System (AS)
- 1875 –UNINETT PCA Policy Statements
- 1371 –Choosing a Common IGP for the IP Internet